

Lesson 5: Statistical Foundations of Machine Learning



Lesson Overview

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.

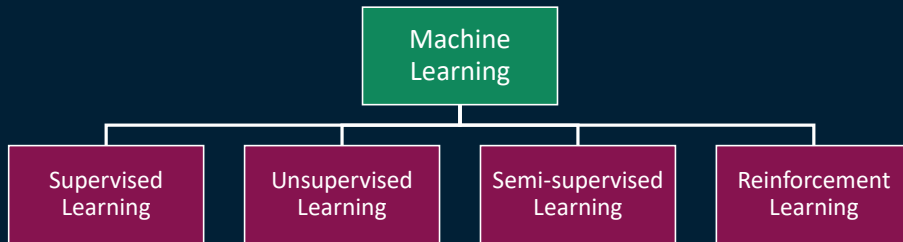


5.1 Overview of Machine Learning



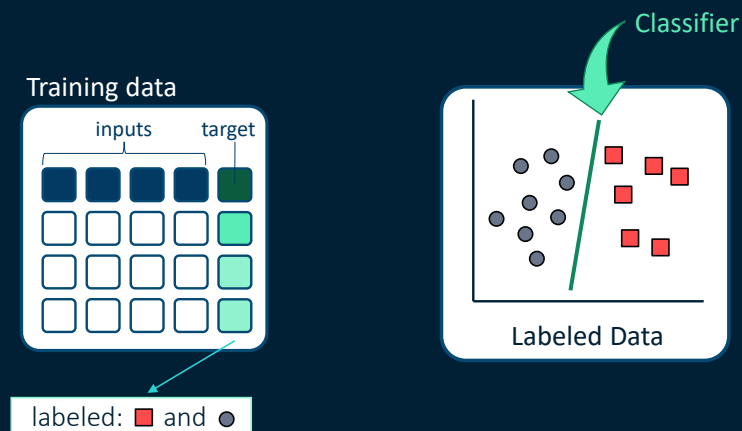
Machine Learning

- Nature of the business problem?
- Type and volume of data?



Depending on the nature of the business problem being addressed, and the type and volume of data, there are different approaches to machine learning. Two of the most widely used machine learning methods are supervised learning and unsupervised learning, but there are also other methods of machine learning. This lesson provides an overview of the most popular types of machine learning: supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning.

Supervised Learning

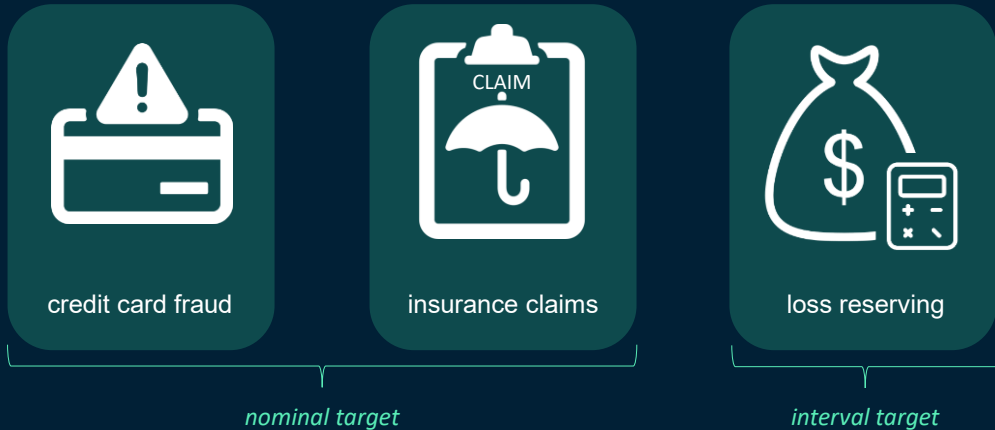


Supervised learning algorithms are trained using labeled examples, such as an input where the desired target is known. Here the value of the target is used as the label for an observation in the table. The learning algorithm receives a set of inputs along with the corresponding correct targets. Target variable could either be categorical or continuous. The algorithm learns by comparing its actual target with predicted target to find errors. It then modifies the model accordingly and produces a classifier.

For example, here you have a binary target: red squares and gray circles, and a linear classifier is represented here with a green line that differentiates red squares with gray circles.

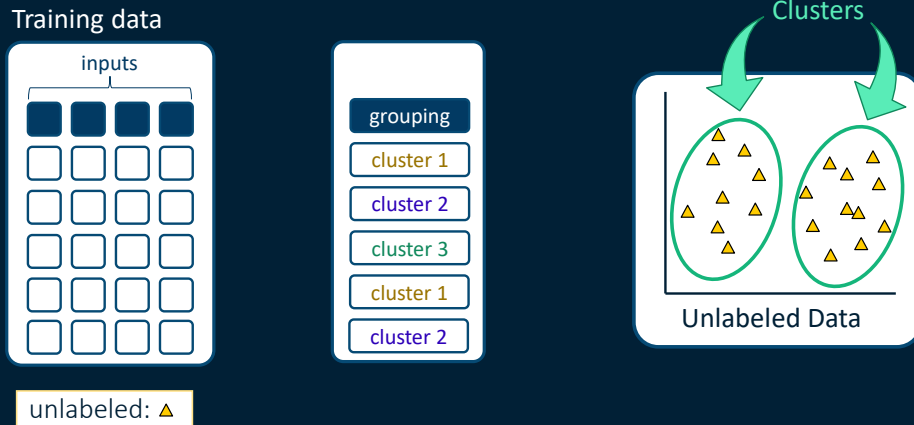
Supervised Learning

used where historical data predict likely future events



Supervised learning is commonly used in applications where historical data predict likely future events. For example, supervised learning can anticipate when credit card transactions are likely to be fraudulent or which insurance customer is likely to file a claim or a loss reserve, which is an estimate of an insurer's liability from future claims that it will have to pay out on.

Unsupervised Learning



Unsupervised learning is used against data that have no historical labels. The system is not told the "right answer." The algorithm must figure out what is being shown. The goal is to explore the data and find some structure within.

Unsupervised learning commonly attempts to group training data set cases based on similarities in input variables. It is a data reduction method because an entire training data set can be represented by a small number of groups. The groupings are known as clusters or segments, and they can be applied to other data sets to classify new cases.

For example, here you have all the observations represented by yellow triangles as no labels are available. A cluster structure is represented here with green ellipses that group the yellow triangles into two mutually exclusive clusters.

Unsupervised Learning

works well on transactional data



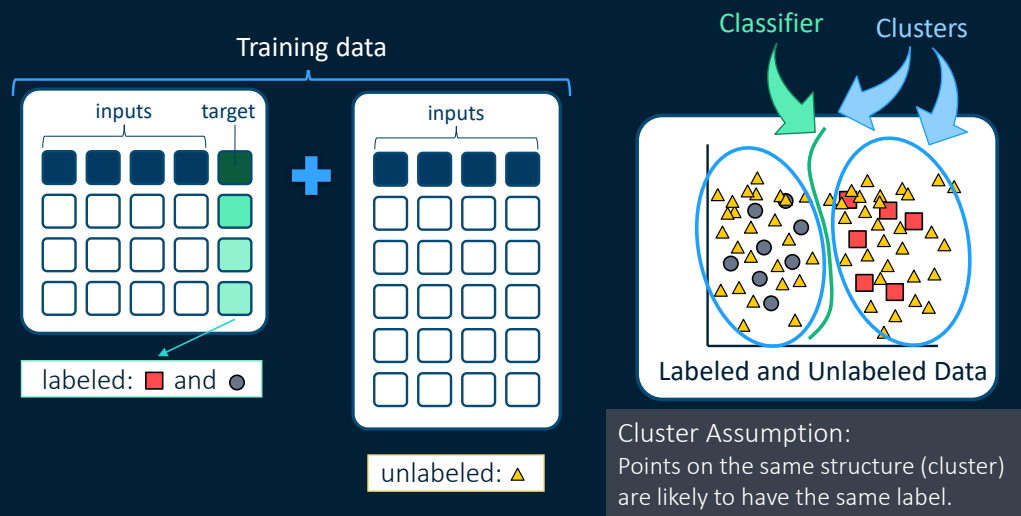
Popular Techniques:

- self-organizing maps
- nearest-neighbor mapping
- k-means clustering
- singular value decomposition

Unsupervised learning works well on transactional data. For example, it can identify segments of customers with similar attributes who can then be treated similarly in marketing campaigns.

Popular techniques include self-organizing maps, nearest-neighbor mapping, k-means clustering, and singular value decomposition. These algorithms are also used to segment text topics, recommend items, and identify data outliers.

Semi-supervised Learning



Semi-supervised learning is used for the same applications as supervised learning. It uses both labeled and unlabeled data for training, typically a small amount of labeled data with a large amount of unlabeled data (because unlabeled data are less expensive and take less effort to acquire). It's kind of a marriage between supervised and unsupervised learning. This type of learning can be used with methods such as classification, regression, and prediction. Input variables are used for distance calculation.

For example, here you have a binary target: red squares and gray circles available for a small number of observations and most of the other observations represented by yellow triangles as no labels are available. Based on the labeled data, a nonlinear classifier is represented here with a green curve that differentiates red squares with gray circles. A cluster structure is represented here with blue ellipses that groups the yellow triangles into two mutually exclusive clusters.

Points on the same structure (typically referred to as a *cluster*) are likely to have the same label. This assumption is often called the *cluster assumption*.

Semi-supervised Learning

useful when the cost associated with labeling is too high



web page
classification



image
recognition



medical
imaging



natural
language
processing



action
recognition

Semi-supervised learning is useful when the cost associated with labeling is too high to allow for a fully labeled training process. The semi-supervised learning algorithm has numerous applications, including web page classification, image recognition, medical imaging, natural language processing, and action recognition.

Reinforcement Learning

useful for sequential decision-making problems



robotics



gaming



navigation
(self-driven car)

The algorithm discovers through **trial and error**
which actions yield the **greatest rewards**.

Reinforcement learning is often used for robotics, gaming, and navigation like a self-driven car. With reinforcement learning, the algorithm discovers through trial and error which actions yield the greatest rewards. Thus, it is mainly utilized for sequential decision-making problems.

More about Reinforcement Learning

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



How Does Machine Learning Work?



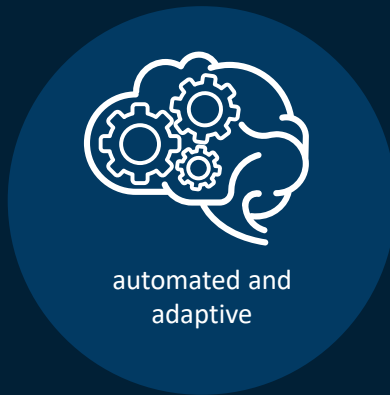
There are many different methods of machine learning. What they all have in common is that they are automated and adaptive. Machine learning techniques are sometimes modeled after the human brain.

I like to use a simple example of distinguishing an apple from an orange. If I want to make an apple pie and I ask you to bring home apples from the market, you need to be able to tell an apple from an orange.

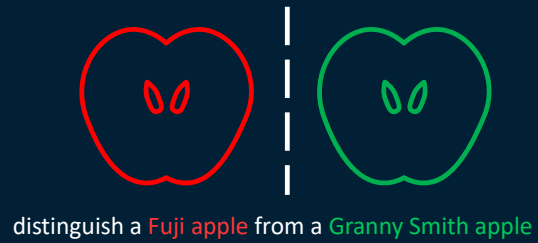
If you come home with oranges, well...best case scenario, there will be no dessert. Luckily, you can do this easily, because over time as a child, you learned to differentiate an apple from an orange based on color, size, smell, and texture. Someone would tell you that yes, that is an apple, or no, that is not an apple. This is called "supervised learning."

Color, size, smell, and texture are all "features" or "attributes" or as statisticians like to call them, independent variables. These features help determine the outcome. In this case, the outcome is binary. It's an apple, or it's not an apple. In your brain, these different attributes are weighted differently. For example, texture and color might be weighted higher than size because it is better at differentiating apples from oranges.

How Does Machine Learning Work?

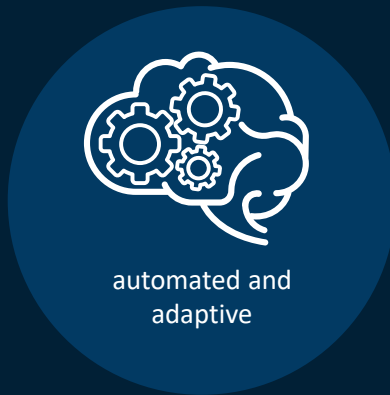


with a higher level of training...

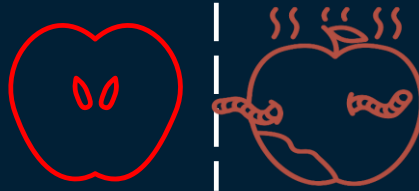


And likewise, with a higher level of training, I can distinguish a Granny Smith apple from a Fuji apple.

How Does Machine Learning Work?



with **more inputs**: *softness, water content,
ethylene gas emitted ...*



distinguish a **ripe apple** from a **rotten apple**

And given more inputs, like softness, water content, and ethylene gas emitted, I could determine not only if an apple is rotten, but how likely it is to become rotten even before it has become brown and squishy.

Question: Machine Learning Methods

Which of the following statements is true regarding machine learning approaches? (Select all that apply)

- a. Unsupervised learning discovers underlying patterns based on input associations.
- b. Supervised learning predicts the output based on input-target associations.
- c. The input data in supervised learning is unlabeled whereas in unsupervised learning the data is labeled.
- d. Reinforcement learning follows a trial-and-error method, and the data is not predefined.

Answer: Machine Learning Methods

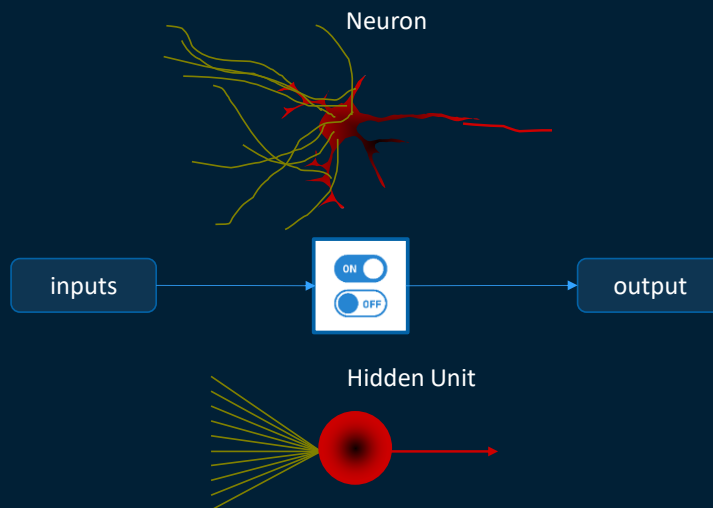
Which of the following statements is true regarding machine learning approaches? (Select all that apply)

- ☒ a. Unsupervised learning discovers underlying patterns based on input associations.
- ☒ b. Supervised learning predicts the output based on input-target associations.
- ☐ c. The input data in supervised learning is unlabeled whereas in unsupervised learning the data is labeled.
- ☒ d. Reinforcement learning follows a trial-and-error method, and the data is not predefined.

ANSWER: a, b and d.

The input data in supervised learning is labeled whereas in unsupervised learning the data is unlabeled. Based on input associations, unsupervised learning discovers underlying data structures such as clusters, whereas based on association of inputs and target, supervised learning predicts the target. With reinforcement learning, the algorithm discovers through trial and error which actions yield the greatest rewards.

Neural Networks

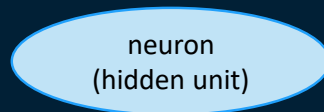


Neural networks, also known as artificial neural networks (ANNs), are at the heart of machine learning. Their name and structure are inspired by the human brain, mimicking the way that biological neurons signal to one another.

A neural network consists of many simple processing elements called neurons or hidden units. Each neuron is connected to other neurons by means of directed communication links, each with an associated weight. A neuron is nothing more than a switch with information input and output. The switch will be activated if there are enough stimuli of other neurons hitting the information input. Then at the information output, a pulse is sent to other neurons.

Neural Networks

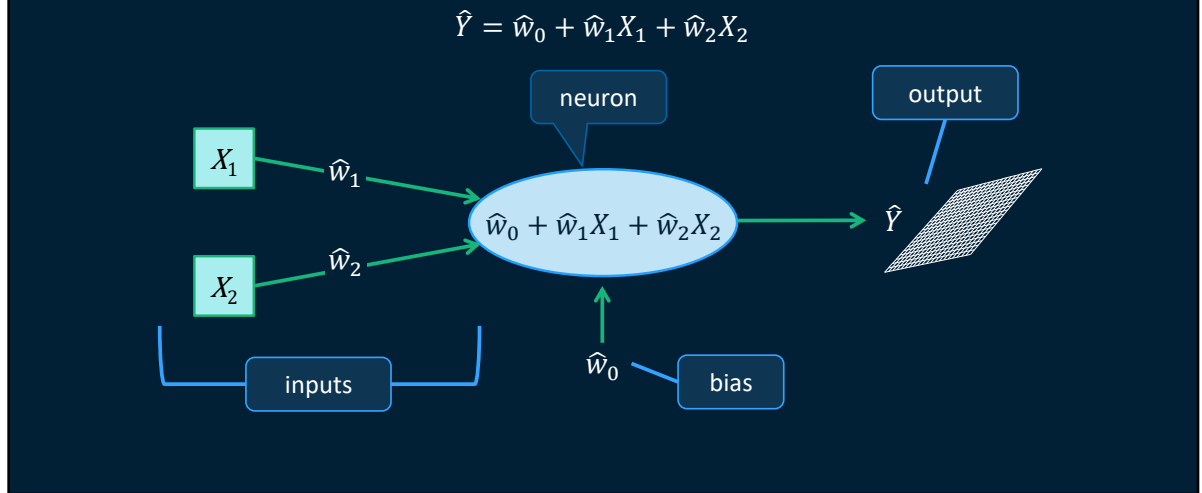
What Is a Neuron?



The basic unit of computation in a neural network is the *neuron*, often called a *hidden unit*. In order to understand neurons, let's look at regression models represented as neural networks.

Neural Networks

Linear Regression Model as a Neural Network

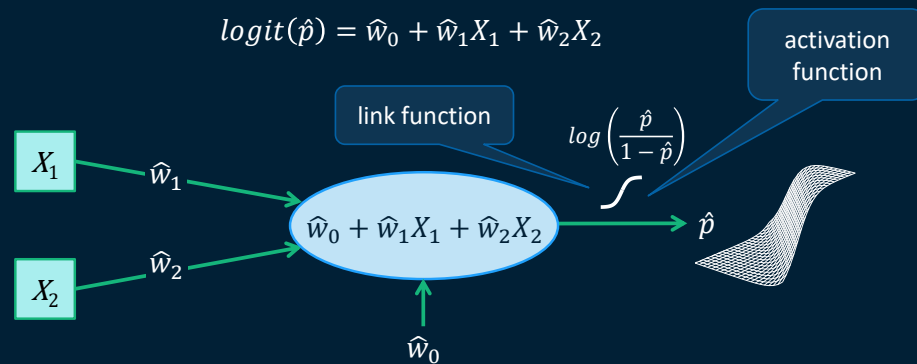


The classic linear regression equation is probably well known to you. Consider a simple linear regression equation with two inputs. The same can be represented in the form of a neural network.

In a neuron, typically, each input (x_i) is first weighted (w_1 and w_2) and then summed. To mimic a neuron's threshold functionality, a bias value (w_0) is added to the weighted sum, predisposing the neuron to give the output value. Here, the bias term is the y-intercept, and the weight associated with each input is the input's slope parameter. A neural network with no hidden layer, only one output neuron, and a linear function of x_i resembles a regression model. In this case, the model is easy to understand and to fit but has a downside – it can model only linear relationships!

Neural Networks

Logistic Regression Model as a Neural Network



A little modification in the previous linear regression model is made. Represented here is a logistic regression model for two inputs X_1 and X_2 with a binary target. An output *activation function* is used to transform the output into a suitable scale for the expected value of the target. In statistics, this function is called the inverse *link function*. For binary targets, the logistic function is suitable because it constrains the output to be between zero and one.

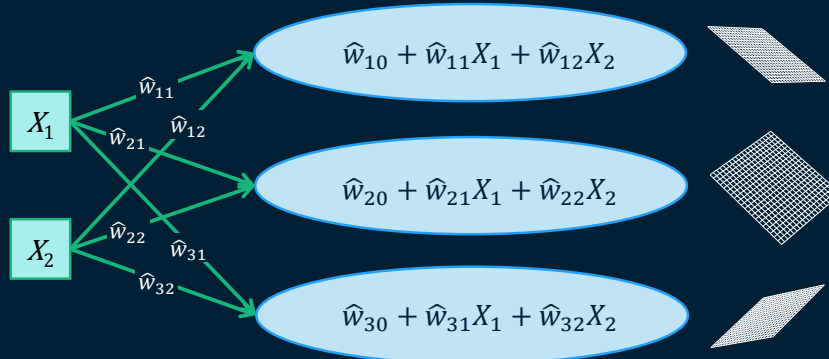
Notice that the neuron is still linear, but as we make this value go through a non-linear function, the overall result is no longer linear. This model is still too simple and might not handle the assumed underlying complexity of the relationship between inputs and output. We can take a step further and deepen the model.

Neural Networks

A Neural Network Model

universal
approximator

3 neurons with linear combination functions

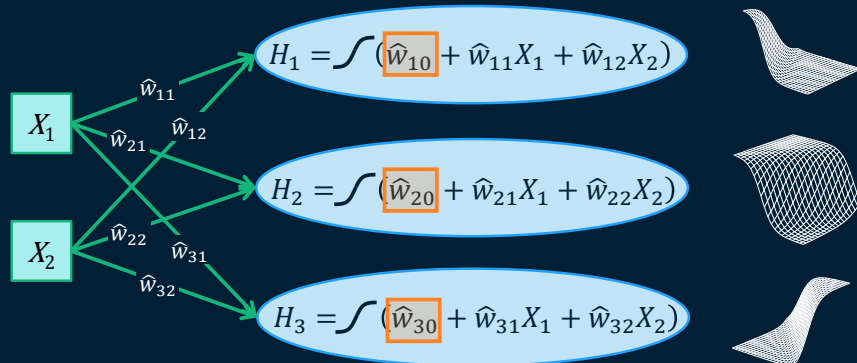


The foremost benefit of neural networks is their unlimited flexibility. A neural network is a *universal approximator*, which means that, with enough neurons and enough time, a neural network can model any input-output relationship, no matter how complex. Let's consider a simple example with three neurons. Each neuron receives a linear combination of input variables, but possibly with different weights. The weights from the linear combinations of the input variables are labeled in the diagram.

Neural Networks

A Neural Network Model

3 neurons with nonlinear activation functions applied

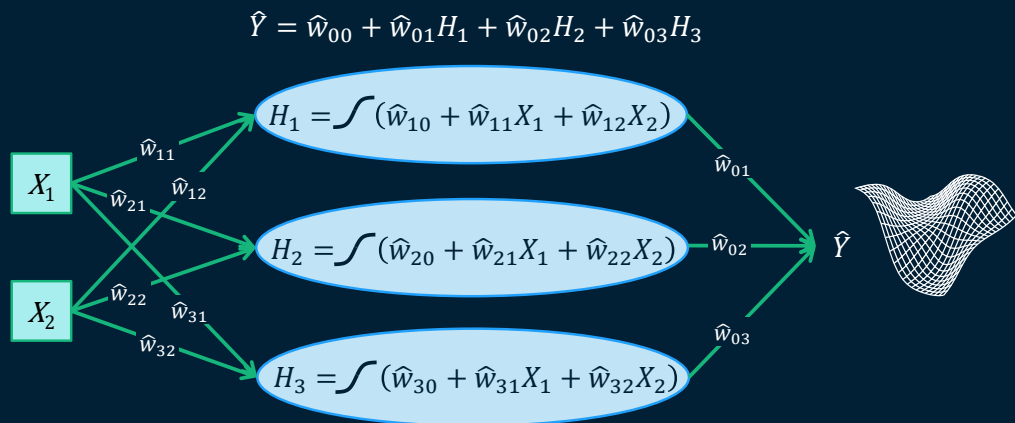


To capture progressively more nonlinearities, linear combinations in each neuron go through a nonlinear *activation function*. An activation function transforms the linear combinations and then converts them to another unit that can then use them as inputs. Different activation functions can be used to produce different nonlinear functions.

When there is a single hidden layer, the hidden units can be thought of as regressions applied to linear combinations of the input variables. In this example, an equation for each of the three hidden units is shown. In each equation, the first w-hat term is the bias term, which is not represented in the diagram. The second subscript on a bias term is always zero.

Neural Networks

A Neural Network Model



Finally, a neural network can be thought of as a regression model on a set of derived inputs, called *hidden units*. The functional form (or the prediction formula) for this neural network model is shown at the top.

Each linear combination of the input variables contains additional weights and a bias term. Remember that the lines in the neural network diagram represent the weights. The weights from the linear combinations of the input variables and the weights from the hidden units are now all labeled in the diagram. For example, the line connecting the hidden unit H_1 to the input x_2 represents the weight $\hat{w}_{12}$. And the line connecting the target to the hidden unit H_1 represents the weight $\hat{w}_{01}$. An output activation function is used to transform the output into a suitable scale for the expected value of the target to produce a nonlinear model.

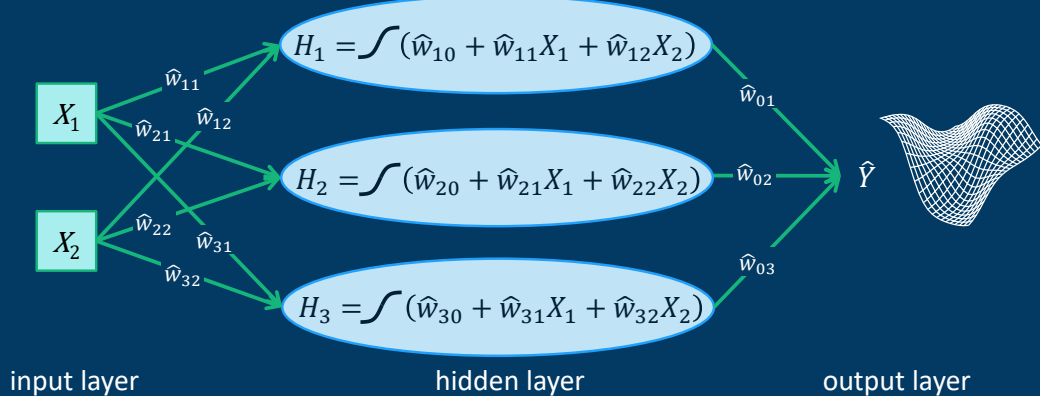
To summarize, each neuron in a network transforms data using a series of computations: a neuron multiplies an initial value by some weight, sums results with other values coming into the same neuron, adjusts the resulting number by the neuron's bias, and then normalizes the output with an activation function.

Neural Networks

A Neural Network Model

$$\hat{Y} = \hat{w}_{00} + \hat{w}_{01}H_1 + \hat{w}_{02}H_2 + \hat{w}_{03}H_3$$

Multilayer Perceptron (MLP)



Represented here is the most widely used neural network known as the *multilayer perceptron* (MLP). An MLP consists of an input layer, one or more hidden layers composed of hidden units, and an output layer.

Question: Neural Networks

Linear regression and logistic regression models can be viewed as neural networks with no hidden layers.

- a. True
- b. False

Answer: Neural Networks

Linear regression and logistic regression models can be viewed as neural networks with no hidden layers.

- ☒ a. True
- ☐ b. False

ANSWER: True

A neural network with no hidden layer, only one output neuron, and a linear function of input variables resembles a regression model.

More Information: Common Algorithms

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Data Preparation for Machine Learning



What **type of data** do
I need for my
machine learning
tasks?

So, by now you might be wondering: *what type of data do I need for machine learning tasks?*

Data Preparation for Machine Learning

Data Arrangement

Long-Narrow

<u>Acct</u>	<u>Type</u>
2133	MTG
2133	SVG
2133	CK
2653	CK
2653	SVG
3544	MTG
3544	CK
3544	MMF
3544	CD
3544	LOC



How should this banking data be arranged?

Machine learning methods require data to have one row per entity.

Short-Wide

<u>Acct</u>	<u>CK</u>	<u>SVG</u>	<u>MMF</u>	<u>CD</u>	<u>LOC</u>	<u>MTG</u>
2133	1	1	0	0	0	1
2653	1	1	0	0	0	0
3544	1	0	1	1	1	1

The data often must be manipulated into a structure suitable for a particular analysis-by-software combination. For example, should this banking data be arranged with multiple rows for each account-product combination (long-narrow form) or with a single row for each account and multiple columns for each product (short-wide form)? For example, account ID 2133 has multiple rows in long-narrow table. It can be arranged in a short-wide form in a single row, but multiple columns of binary type. Machine learning methods require data to have one row per entity.

Data Preparation for Machine Learning

Rolling Up Cross-Sectional Data

 *make comparisons at a single point*

<u>HH</u>	<u>Acct</u>	<u>Sales</u>		<u>HH</u>	<u>Acct</u>	<u>Sales</u>
4461	2133	160	{	4461	2133	?
4461	2244	42		4911	3544	?
4461	2773	212		5630	2496	?
4461	2653	250		6225	4244	?
4461	2801	122				
4911	3544	786	{			
5630	2496	458	{			
5630	2635	328	{			
6225	4244	27	{			
6225	4165	759				

Business requirements often dictate rolling up accounts from a single household into a single record (case). This process usually involves creating new summary data. How should the sales figures for multiple accounts in a household be summarized? Using the sum, the mean, the variance, or all three?

Given here is an example of *cross-sectional* studies that make comparisons at a single point in time. Next is an example of *longitudinal* studies that make comparisons over time.

Data Preparation for Machine Learning

Rolling Up Longitudinal Data

 *make comparisons over time*

How should flying mileage be consolidated?



	<u>Frequent Flier</u>	<u>Month</u>	<u>Flying Mileage</u>	<u>VIP Member</u>
{	10621	Jan	650	No
	10621	Feb	0	No
	10621	Mar	0	No
	10621	Apr	250	No
{	33855	Jan	350	No
	33855	Feb	300	No
	33855	Mar	1200	Yes
	33855	Apr	850	Yes

In some situations, it might be necessary to roll up longitudinal data into a single record for everyone. Longitudinal data, sometimes referred to as panel data, track the same sample at different points in time. For example, suppose an airline wants to build a prediction model to target current frequent fliers for a membership offer in the "Very Important Passenger" club. One record per passenger is needed for supervised classification. How should the flying mileage be consolidated if it is to be used as a predictor of club membership?

Data Preparation for Machine Learning

Joining Tables

Rolled-up flying mileage data

<u>Frequent Flier</u>	<u>Flying Mileage</u>	<u>VIP Member</u>
10621	900	No
33855	2700	Yes

Demographic data

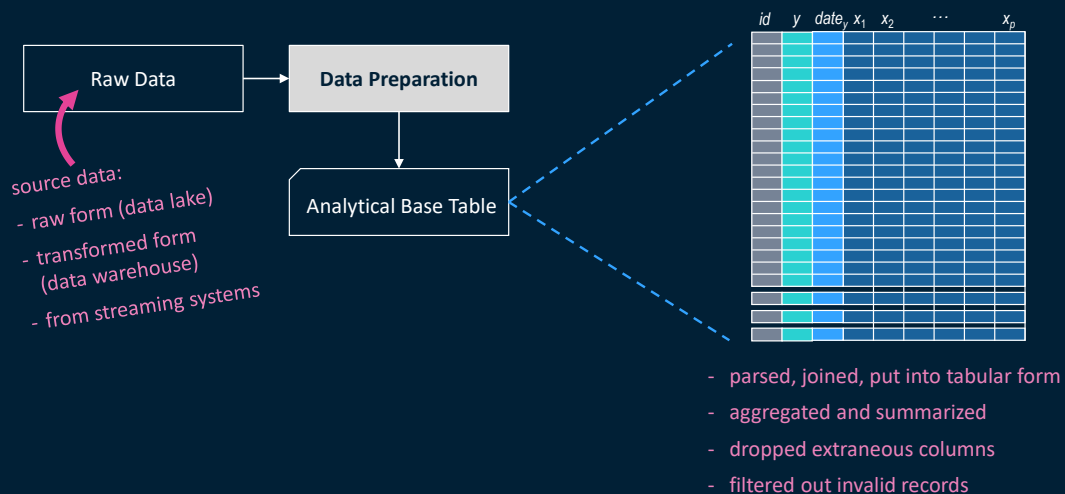
<u>Frequent Flier</u>	<u>Age</u>	<u>Income Level</u>	<u>Home Owner</u>
10621	45	Med	Yes
33855	28	High	No



<u>Frequent Flier</u>	<u>Flying Mileage</u>	<u>VIP Member</u>	<u>Age</u>	<u>Income Level</u>	<u>Home Owner</u>
10621	900	No	45	Med	Yes
33855	2700	Yes	28	High	No

In addition to rolling-up, we need to join several tables to stitch together different types of data. For example, after flying mileage is consolidated as the sum of the four months, it can be joined with the demographic data. Frequent flyer column value can be used as a key to match the two tables.

Data Preparation for Machine Learning

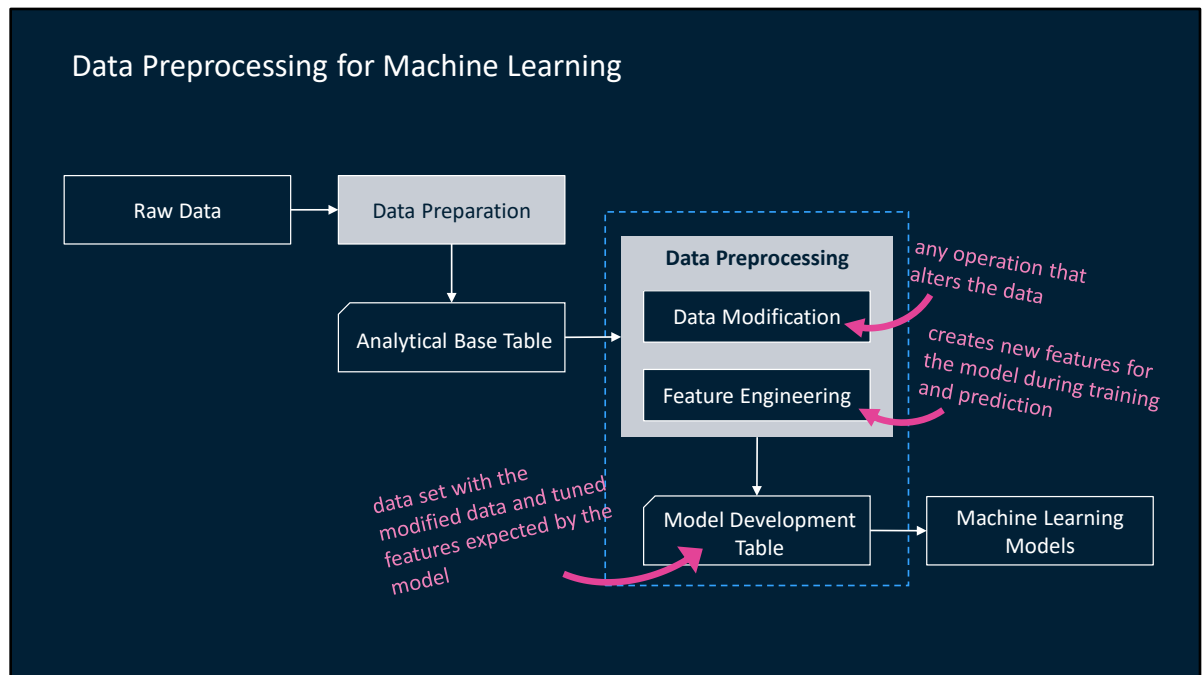


As you've just seen in the previous examples, the data used for machine learning involves a grueling data preparation phase. Data preparation is the process of converting *raw data* into prepared data commonly known as *analytical base table*.

Raw data refers to the data in its source form, without any prior preparation for machine learning. This type of data might be in its raw form (in a data lake) or in a transformed form (in a data warehouse). In addition, data sent from streaming systems that eventually call machine learning models for predictions is data in its raw form only.

Analytical Base Table (ABT) is usually a wide table that refers to the data set in the form ready for your machine learning task. Data sources have been parsed, joined, and put into a tabular form. Data has been aggregated and summarized to the right granularity—for example, each row or an ID column in the data set represents a unique customer, and each column represents summary information for the customer, like the total spent in the last two months. In the case of supervised learning tasks, the target feature is present along with a target date, which is optional. Extraneous columns have been dropped, and invalid records have been filtered out.

Data Preprocessing for Machine Learning



Even when you get to the analytical base table, you are yet not ready to apply machine learning models. The data now need to be preprocessed for achieving certain modeling requirements to yield a model development table. The model development table refers to the data set with the modified data and tuned features expected by the model.

Data modification is a broad preprocessing category. Any operation that alters the data or data roles can be considered as a modification. *Feature engineering* is another broad preprocessing category that performs certain machine learning-specific operations on the columns in the prepared data set and creating new features for your model during training and prediction. Data preprocessing examples include case filtering, replacement of values, imputation, scaling numerical columns to a value between 0 and 1, truncating values, and one-hot-encoding categorical features.

Data preprocessing is discussed in detail in the next section.

Question: Data Preparation and Data Preprocessing

Which of the following statements is True regarding machine learning data?
(Select all that apply.)

- a. Source data yields a **raw table** without any prior preparation.
- b. Data preparation yields an **analytical base table** in the form ready for your machine learning task, such as feature engineering.
- c. Data preprocessing yields a **model development table** that is suitable to apply to machine learning models.
- d. Machine learning data is generally clean and does not require any preparation and processing.

Answer: Data Preparation and Data Preprocessing

Which of the following statements is True regarding machine learning data?
(Select all that apply.)

- ☒ a. Source data yields a **raw table** without any prior preparation.
- ☒ b. Data preparation yields an **analytical base table** in the form ready for your machine learning task, such as feature engineering.
- ☒ c. Data preprocessing yields a **model development table** that is suitable to apply to machine learning models.
- ☐ d. Machine learning data is generally clean and does not require any preparation and processing.

ANSWER: a, b, and c.

Raw table refers to the data in its source form. *Analytical base table* is usually a wide table that refers to the data set in the form ready for your machine learning task. The data now need to be preprocessed for achieving certain modeling requirements to yield a *model development table*. Machine learning data is generally dirty and thus involves grueling preparation and processing phases.

5.2 Data Pre-processing for Machine Learning Models



Data Difficulties and Modeling Issues



- Visualizing big data
- Processing noisy data
- Handling high dimensional data
- Dealing with distinct variability in inputs
- Using available data judiciously
- Adjusting model flexibility and complexity
- Understanding learning and tuning of models
- Making models explainable

Data Difficulties
(this section)

Modeling Issues
(next section)

The benefits of machine learning translate to innovative applications that can improve the way processes and tasks are accomplished. However, despite its numerous benefits, there are still risks and challenges. Listed here are some of the challenges that we encounter in data mining and machine learning.

The first four: visualizing high-dimensional data, processing noisy data, handling too many variables, and dealing with distinct variability in inputs, are essentially related with machine learning data and will be discussed in this section of the lesson.

The remaining four: using available data economically, addressing model flexibility and complexity, finding best values of parameters and making models explainable, are related with core modeling and will be discussed in the next section of the lesson.

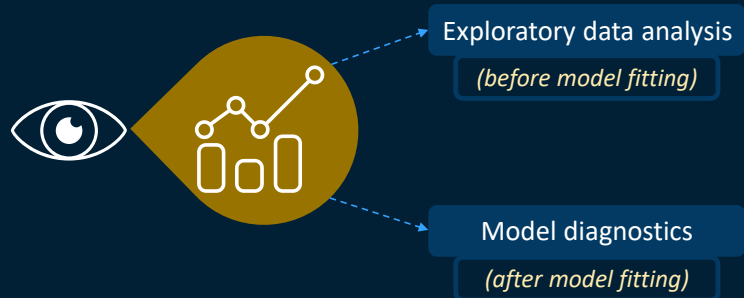
Data Visualization



- ➡ • **Visualizing big data**
 - Processing noisy data
 - Handling high dimensional data
 - Dealing with distinct variability in inputs
 - Using available data judiciously
 - Adjusting model flexibility and complexity
 - Understanding learning and tuning of models
 - Making models explainable

Let's begin our discussion of data difficulties with visualizing big data.

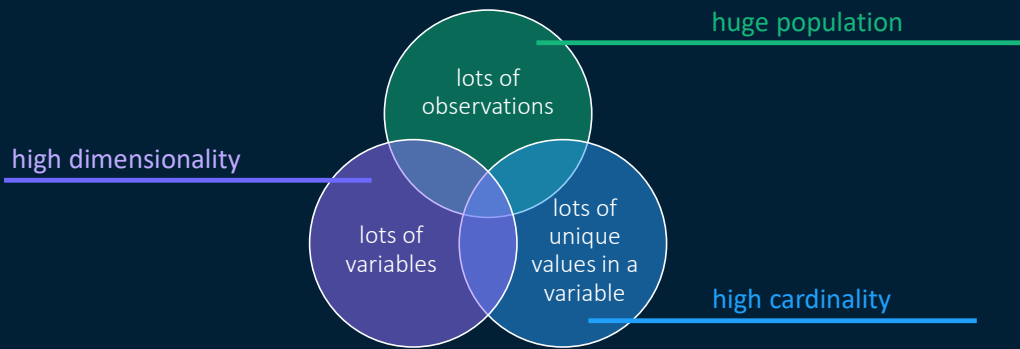
"A picture is worth a thousand words."



A picture is worth a thousand words – especially when you are trying to understand and discover insights from data. It is much easier to understand information in a visual than in a large table with lots of rows and columns. You can choose the most appropriate visualization by understanding the data and its composition, what information you are trying to convey to your audience, and how viewers process visual information.

You need effective data visualizations while exploring the data before fitting models. You also need effective data visualizations for model diagnostics after fitting models. Visualizing your data can be both fun and challenging.

Big Data Visualization Challenges



Big data brings new challenges to visualization because of the speed, size, and diversity of data that must be considered. To create meaningful visuals of your data, consider the following questions. Do you have a lot of observations (or *rows*)? Do you have a lot of variables (or *columns*)? Do your variables have a lot of unique values? Having lots of observations, lots of variables, or lots of unique values in a variable is a challenge for visualization.

Large studies are generally more reliable than smaller studies, and there is a growing interest in the analysis of *huge population* sizes that integrates information from millions of cases or different data sources, or both. The *dimension* of your data refers to the number of input variables. If you have a lot of variables, we call it *high dimensionality*. *Cardinality* is the uniqueness of data values contained in a column. *High cardinality* means there is a large percentage of unique values (such as bank account numbers, because each item should be unique). Low cardinality means a column of data contains a large percentage of repeat values (such as a "gender" column). Population size, dimensionality, and cardinality composition play an important role when selecting graphs to represent your data.

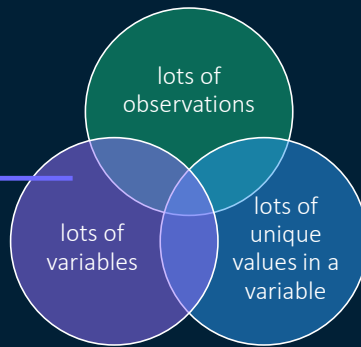
Big Data Visualization Challenges

high dimensionality

- multidimensional space
- presentation of variable associations
- exhibiting relative importance of variables

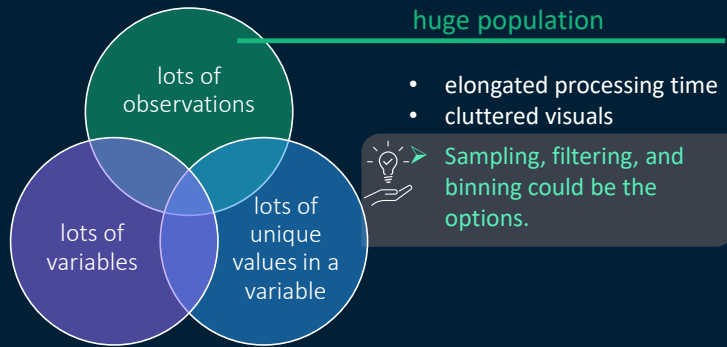


➤ Selecting an appropriate visual is the solution.



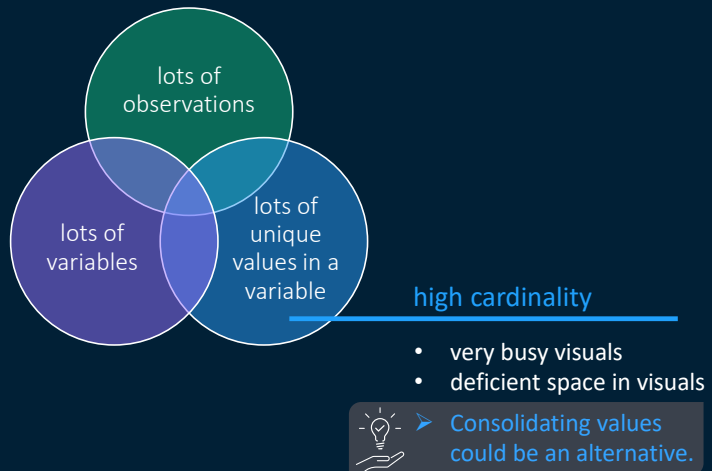
The analysis of high-dimensional data offers a great challenge because the human intuition about geometry of high dimensions fails. One of the biggest challenges in data visualization is to find general representations of data that can display the multivariate structure of more than two variables. Appropriate visuals are especially helpful when you're trying to find relationships among hundreds or thousands of variables to determine their relative importance – or if they are important at all.

Big Data Visualization Challenges



Extremely large populations for visualization pose risk of slower execution. The visuals might be messy and cluttered with countless observations. For example, what if you have a billion rows in a data set and want to create a scatter plot on two measures? It would be impossible to see so many data points. And the application creating the visual might not be able to plot a billion points in a timely or effective manner. In such a scenario perhaps, you can explore the data on a workable sample or use *binning* (the grouping together of data) on both axes so that you can effectively visualize the big data. When working with large amounts of data, being able to quickly and easily filter your data is important. What if you want to view data only for a certain region, product line, or some other variable?

Big Data Visualization Challenges



Another concern with big data is cardinality because the data might have many unique values per column. If there are too many columns in your bar chart, you cannot see the labels for each bar, and the graph becomes less meaningful. The resulting visuals might be very busy and chaotic. If you are working with high cardinality data, one challenge is displaying output in a way that's not overwhelming. You might need to collapse and condense the data while still providing graphs and charts that decision makers are accustomed to seeing.

Visualization Examples

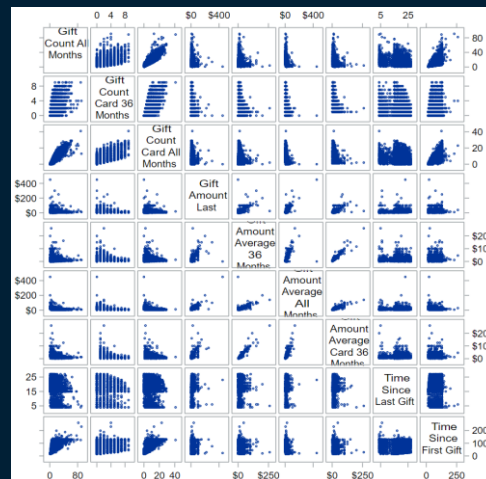
Traditional Statistical Plots

As the number of variables and observations increases...

...the plot can become more cluttered



Scatter Plot Matrix



Traditional tools, might take significant time.



When analyzing relationships between many numeric variables, scatter plots and correlation coefficients can quickly uncover patterns to a subset of interesting relationships. Suppose you want to examine the relationship between nine continuous variables. Of course, the first choice would be to create a scatter plot matrix that displays every possible pairing of the measures that are assigned to the visualization. However, as the number of variables and observations increases, the plot can become more cluttered. Also, in a big data scenario, traditional tools, or algorithms, might take significant time to return results.

Given this scatter plot, can you determine the strength of correlation between pairs of variables? Is there an efficient way to analyze the relationships between these continuous variables while accounting for data size and ease of interpretation?

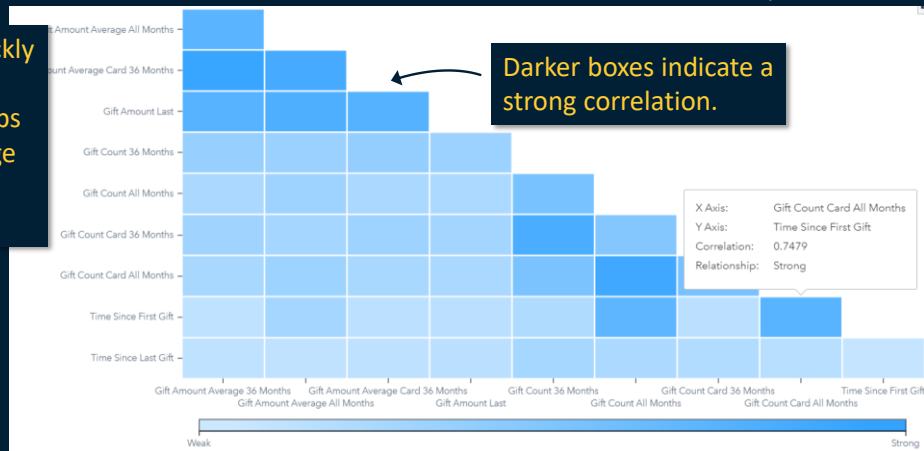
Visualization Examples

Think beyond Traditional Statistical Plots!

Correlation Matrix

Might be insufficient to show many pairs of relationships.

Use to quickly assess relationships among large number of variables.



Yes, there are new sophisticated tools and techniques that can address the challenges associated with visualizing big data. The output shown here is from SAS Visual Analytics that uses intelligent autocharting to help business analysts and nontechnical users easily visualize their data.

So, instead of using the traditional scatter plot matrix, you use a correlation matrix plot, which makes it easy to quickly assess relationships among large number of variables. It also shows how strong the relationship is between variables. A color-coded correlation matrix quickly identifies strong and weak relationships between the variables. Darker boxes indicate a strong correlation and lighter boxes indicate a weaker correlation. Also, if you hover your mouse pointer over a box, a summary of the relationship is shown.

So, even if you have more than nine variables, this chart is still relevant. However, a scatter plot matrix might be insufficient to show many pairs of relationships.

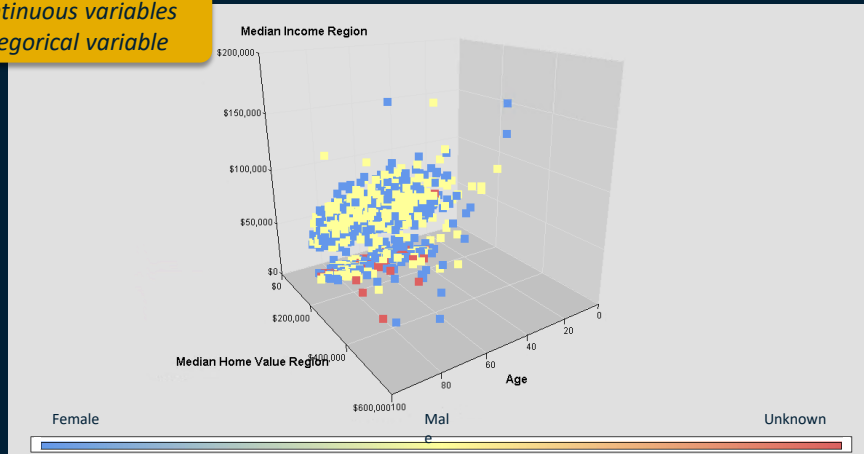
Visualization Examples

Traditional Statistical Plots

3 continuous variables
1 categorical variable

Color Coded 3-D Scatter Plot

Can be challenging to interpret!



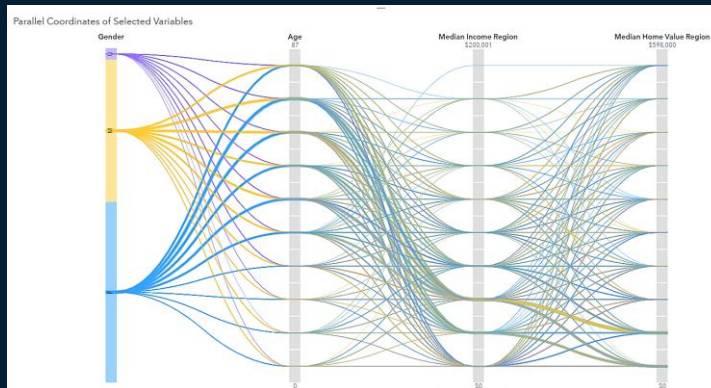
Consider another scenario where you want to explore the relationship between three numeric variables and a categorical variable. The first visualization technique that you might think of is a 3-D scatter plot, which can be used to show the relationship between three numeric variables. A fourth variable can be added by matching the color of the markers, as shown in this chart where we are trying to understand the relationship between median income region, median home value region, age, and gender. Needless to say, a three-dimensional plot is more challenging to interpret than a 2-dimensional plot. Also, if you needed to add few more variables in this exploration, a 3-D scatter plot wouldn't allow this.

Visualization Examples

Think beyond Traditional Statistical Plots!

efficient for analyzing
multivariate numerical
data

Parallel Coordinate Plot



- use to compare multiple numerical variables with
 - different magnitudes and
 - different units of measurement

In big data, you quite often come across situations where you need to explore relationships between many variables (say, more than three). Well, the good news is that there are efficient visualization techniques for analyzing multivariate numerical data. One such technique is the *parallel coordinate plot*, as shown in this example.

A parallel coordinate plot displays data as lines moving through categories and binned measures. These lines are color coded and often are called *polylines*. The thickness of a polyline indicates the relative number of observations in that bin. You can restrict the active polylines to one or more bins in order to focus on only the data that interests you, which means that the user can interact with the plot.

A parallel coordinate plot enables you to compare many quantitative variables together, looking for patterns and relationships between them. They are appropriate for simultaneously comparing multiple numerical variables when those variables have different magnitudes (or different scales) and different units of measurement. The idea is to find patterns, similarities, clusters, and positive, negative, or no particular relationships in multidimensional data sets. And most importantly, you can explore the relationship between multiple variables while using a two-dimensional plot.

More about Visualization Examples

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Data Visualization Challenges

Which of these terms describes a situation with ***lots of unique values in a variable***?

- a. Huge population
- b. High dimensionality
- c. High cardinality
- d. Low variability

Answer: Data Visualization Challenges

Which of these terms describes a situation with ***lots of unique values in a variable***?

- a. Huge population
- b. High dimensionality
- ☒ c. High cardinality
- d. Low variability

The correct answer is c.

When you have lots of unique values in a variable, it is typically termed as *high cardinality*. Huge population is a situation with lots of observations. High dimensionality is a situation with lots of variables. Low variability means that the values are more consistent.

Dirty Data: Errors, Missing Values, and Outliers



- Visualizing big data
- ➔ • **Processing noisy data**
- Handling high dimensional data
- Dealing with distinct variability in inputs
- Using available data judiciously
- Adjusting model flexibility and complexity
- Understanding learning and tuning of models
- Making models explainable

Data encryption errors, missingness, and outliers are commonly present in noisy data.

Dirty Data: Errors, Missing Values, and Outliers



Checking Account	No. of Checking Accounts	Average Daily Balance	Non Sufficient Funds	Direct Deposit for Sure	Savings Account	Closing Balance
<u>cking</u>	<u>#cking</u>	<u>ADB</u>	<u>NSF</u>	<u>dirdep</u>	<u>SVG</u>	<u>bal</u>
Y	1	468.11	1	1876	Y	1208
Y	1	68.75	0	0	Y	0
Y	1	212.04	0	6		0
.	.	.	0	0	Y	4301
y	2	585.05	0	7218	Y	234
Y	1	47.69	2	1256		238
.	1	4687.7	0	0		0
Y	.	.	1	0	Y	1208
.	.	.	.	1598		0
.	1	0.00	0	0		0
Y	3	89981.12	0	0	Y	45662
Y	2	585.05	0	7218	Y	234

Shown here is a simple example of dirty data from banking domain. Variable labels are listed for better understanding. You will need them for upcoming student activities.

Inadequate data scrutiny is a common oversight. Errors, outliers, and missing values must be detected, investigated, and corrected (if possible). The basic data scrutiny tools are raw listings, if/then subsetting functions, exploratory graphics, and descriptive statistics such as frequency counts and minimum and maximum values.

Question: Errors

What do you see in the data below that looks wrong or even just out of place? There really are no wrong answers here. No matter how trivial, what should be investigated?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Possible Answer: Errors

There is a lowercase y in column 1 [R5C1] versus all the uppercase Ys.

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Live Web students: Type in the chat what you think should be investigated.

Possible Answer: Errors

In the sixth row, values in C3-C7 are not in line with the others.

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Live Web students: Type in the chat what you think should be investigated.

Possible Answer: Errors

For the **SVG** column, a row is blank [R6C6] when its corresponding **bal** column has a nonzero number.

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Live Web students: Type in the chat what you think should be investigated.

Possible Answer: Errors

Rows 5 and 12 are the same except for the lowercase y in R5C1. Are these the same observation where someone corrected the y but forgot to remove the original row?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Live Web students: Type in the chat what you think should be investigated.

Possible Answer: Errors

What does a 2 in the **NSF** column [R6C4] mean? Is NSF (non-sufficient funds) binary?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Live Web students: Type in the chat what you think should be investigated.

Errors



Types of Errors

- Impossible values
- Impossible combinations of values
- Inconsistent coding
- Coding mistakes
- Repeated records and more ...

Detection requires
persistence, creativity,
and **domain knowledge!**



Detection of such errors as impossible values, impossible combinations of values, inconsistent coding, coding mistakes, and repeated records require persistence, creativity, and domain knowledge. You have already seen most of these errors in the preceding activity.

Demo: Modifying and Correcting Data

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Missing Values

Are there any missing values?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Missing values occur for a variety of reasons. They often represent unknown but knowable information. Structural missing data represent values that logically could not have a value. Missing values are often coded in different ways - sometimes by a question mark, or -9999, sometimes by "n/a" or by some other specific number or character; and sometimes miscoded as zeros. The reasons for the coding and the consistency of the coding must be investigated.

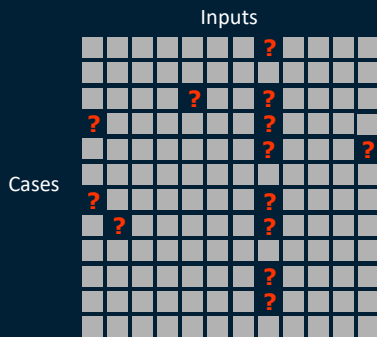
Answer: Missing Values

Are there any missing values?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Missing values occur for a variety of reasons. They often represent unknown but knowable information. Structural missing data represent values that logically could not have a value. Missing values are often coded in different ways - sometimes by a question mark, or -9999, sometimes by "n/a" or by some other specific number or character; and sometimes miscoded as zeros. The reasons for the coding and the consistency of the coding must be investigated.

Missing Data



- Missing data problems
 - Missing data analysis strategies
 - Missing data mechanisms
 - Missing data causes
- } (Self-Study)

Nearly all the real-world data have missing values, and dealing with missing values or incomplete observations is critical in any data mining and machine learning analysis. The term missing data is defined here as a statistical problem characterized by an incomplete data matrix where there's one or multiple variables where certain observations are missing for those variables. It is important to understand:

What are problems of missing data?

How to handle missing values?

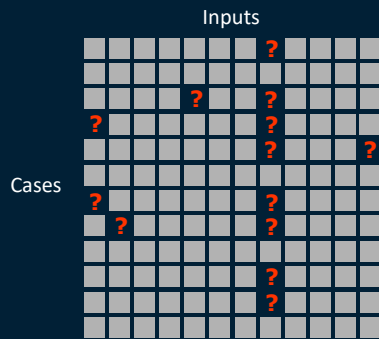
What are the types of missingness? And why is your data missing? (You can learn the last two in a self-study section of this lesson.)

Missing Data Mechanisms

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Missing Data Problems



Major problems caused by missing values:



- Parameter bias
 - overestimation or underestimation



- Inferential error
 - prediction error or hypothesis testing errors

missing data problems (Newman 2014)

The two major problems caused by missing data are parameter bias and inferential error.

Bias refers to the systematic overestimation or underestimation of a parameter (such as underestimated mean, correlation, or regression coefficient). Missing data bias typically occurs when the missingness mechanism is systematic or nonrandom - under MAR or MNAR.

Error refers to prediction error or hypothesis testing errors of inference, such as Type I error (also called *false positive*) and Type II error (or *low power*, also called *false negative*).

More about Missing Data Problems

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Missing Data Causes

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Missing Value Representation

How are missing values in a numeric variable represented in SAS?

- a. By a blank (' ')
- b. By a period (.)
- c. By a comma (,)
- d. By a semicolon (;)

Answer: Missing Value Representation

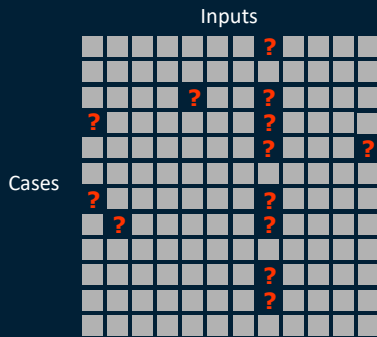
How are missing values in a numeric variable represented in SAS?

- a. By a blank (' ')
- ☒ b. By a period (.)
- c. By a comma (,)
- d. By a semicolon (;)

ANSWER: b

In SAS, numeric missing values are represented by a period (.), whereas character missing values are represented by a blank (' ').

Analysis Strategies for Missing Data



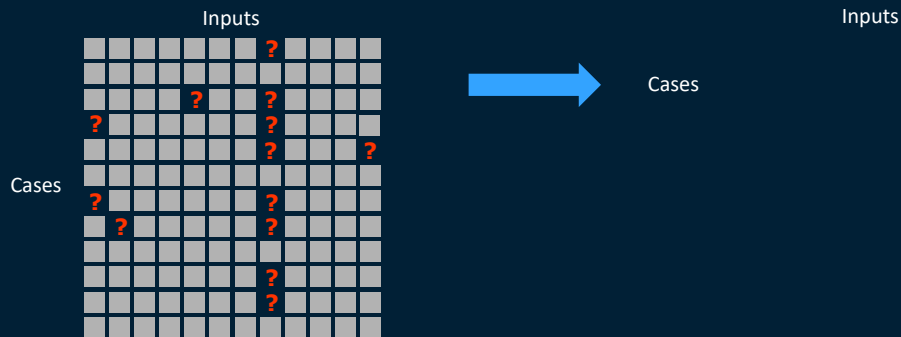
Analysis strategies for handling missing value

- Complete-case analysis (CCA)
- Imputation
- Missing indicators

There are predominantly three analysis strategies for handling missing values.

Analysis Strategies for Missing Data

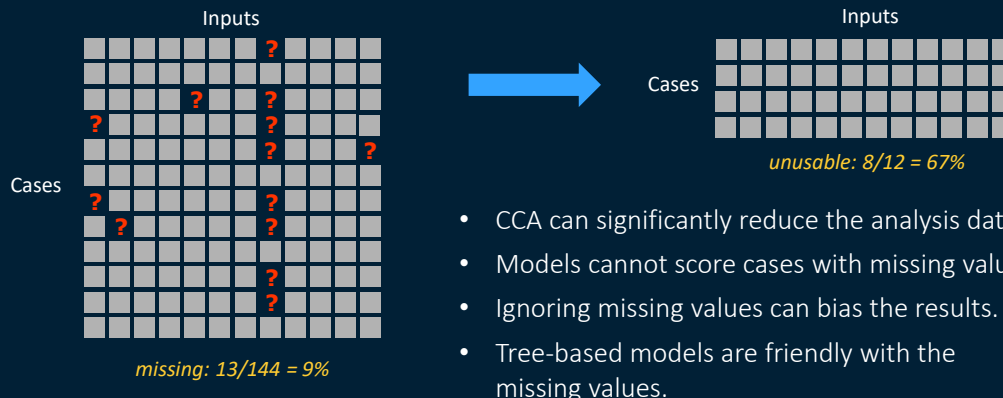
Complete-Case Analysis (CCA)



The simplest strategy is 'do nothing' when you encounter missing values. However, missing values can be theoretically and practically problematic for many machine learning models such as **Neural Network** or **Regression** or **SVM**. This will lead you to use only the cases that have complete records in the analysis, commonly known as *complete-case analysis*.

Analysis Strategies for Missing Data

Complete-Case Analysis (CCA)



- CCA can significantly reduce the analysis data.
- Models cannot score cases with missing values.
- Ignoring missing values can bias the results.
- Tree-based models are friendly with the missing values.

The main difficulty with this strategy is practical. Even a small amount of missing values in a high dimensional data set can cause a disastrous reduction in data. In the above example, only 13 of the 144 values (9%) are missing, but a complete-case analysis would use only four cases—only a third of the data set. If there are very few cases with missing values, this is a viable option.

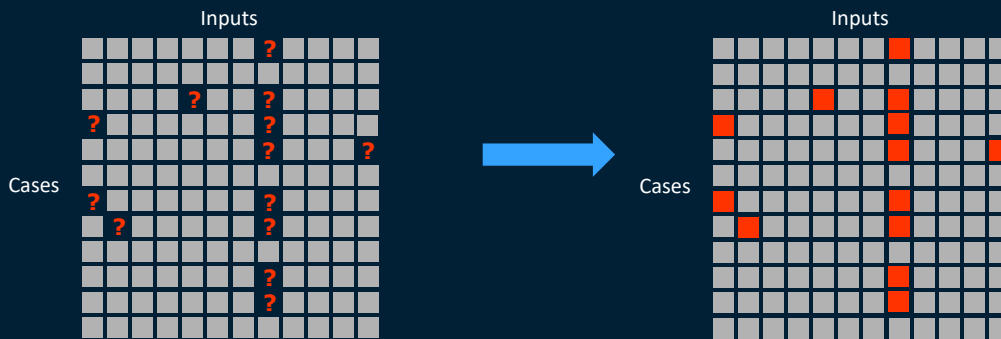
Another problem relates to model deployment. How would a model built on the complete cases score a new case if it had a missing value? If there is missingness in your training data, it is very likely that your scoring data or the new data would also have, ideally, a similar type of missingness, but in limited amount. Thus, inefficient scorability.

Rejecting all incomplete observations might also bias the sample, because observations that have missing values might have other things in common as well. For example, if the "missingness" is related to the inputs or to the target, then ignoring missing values can bias the results. Omitting the missing term from the parametric equation usually produces an extremely biased prediction.

Nevertheless, some other machine learning models such as **decision tree**, **forest**, and **gradient boosting** can use missing values to their advantage. For example, missing values can be treated as legitimate values while training these models. So missingness is not always awkward.

Analysis Strategies for Missing Data

Imputation



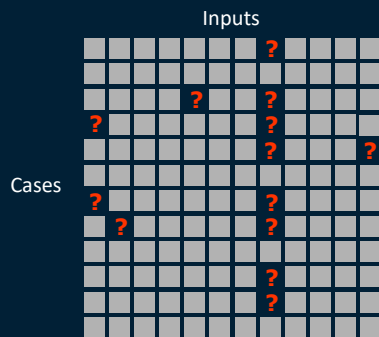
- Fixed-value imputation (Synthetic distribution)
- Tailored-value imputation (Estimation)
- Cluster imputation

The primary remedy for missing values in most of the machine learning models is a missing value replacement strategy called *imputation*. Fill in the missing values with some **reasonable** value and then run the analysis on the full (filled-in) data. Imputation is an important aspect of data preprocessing that has the potential to make (or break) your model. The goal of imputation is to replace missing values with values that are close to what the missed value might have been. If successful, you will have more data with which the model can learn. However, you could introduce serious bias and make learning more difficult. The bottom line is that whether you should use imputation for missing values is completely dependent on your data and what the missing value represents. Be diligent!

Missing value replacement strategies fall into one of three categories: fixed-value imputation, tailored-value imputation, and cluster imputation.

Analysis Strategies for Missing Data

Fixed-Value Imputation

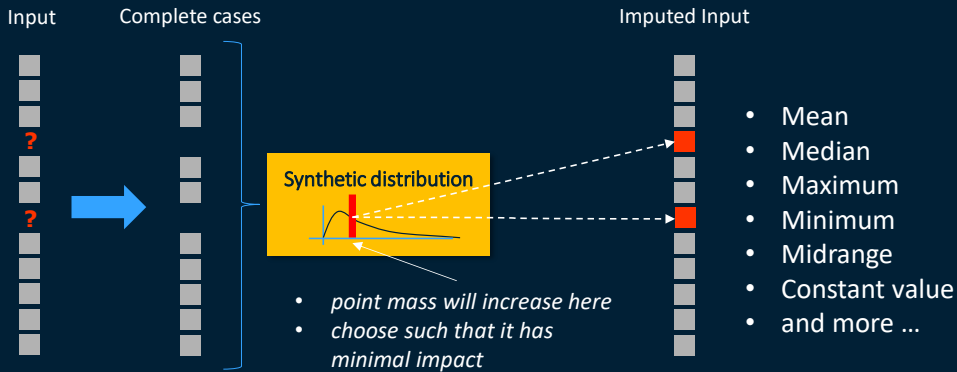


The simplest types of imputation methods fill in the missing values with a fixed value. For example, you can impute the missing values in a survey with a "not answered" category. That is, use a one-size-fits-all approach to handle missing values. Fixed value imputation is a general method that works for all data types and consists of substituting the missing value with a fixed value.

As an illustration, consider the variable in the first column. Out of 12 values, two are missing.

Analysis Strategies for Missing Data

Fixed-Value Imputation



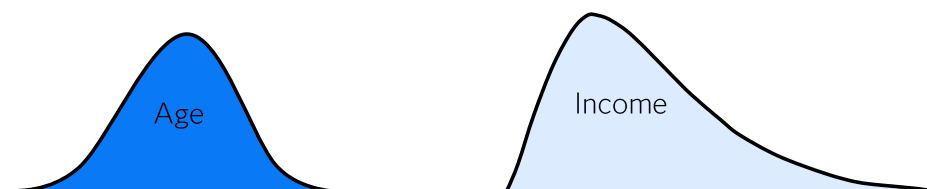
Taking one variable at a time, there are 10 complete observations. The simplest types of imputation methods fill in the missing values with the mean of the complete cases. Thus, the two missing values are replaced with the arithmetic mean computed on these 10 observations. In any case with a missing input measurement, the missing value is replaced with a fixed number. As a result, the point mass will increase at the selected fixed number in the synthetic distribution; thus, the choice of the fixed value is not arbitrary. Ideally, it should be chosen to have minimal impact on the magnitude of an input's association with the target. With many modeling methods, this can be achieved by locating the point mass at the input's mean value.

If you have determined that imputation might be beneficial for your data, there are several options to choose from. We will not be going into detail about the types of imputation available, as the type you use will be highly dependent on your data and goals. The examples listed here give you a glimpse of the possibilities.

The missing values of categorical variables could be treated as a separate category. For example, *type of residence* might be coded as *own home*, *buying home*, *rents home*, *rents apartment*, *lives with parents*, *mobile home*, and *unknown*. This method would be preferable if the missingness is itself a predictor of the target.

Question: Mean or Median Imputation

Age and **Income** are two variables that have missing values. You can use either mean or median imputation, but each only once. Variable distributions are given below. Which one you would use and why?

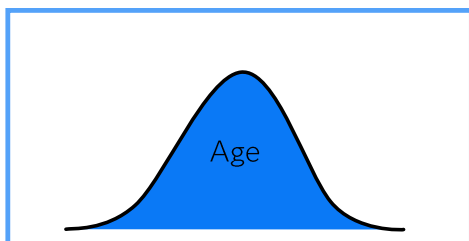


Because the **Age** variable is nearly normal, mean imputation would work. Nonetheless, for a normally distributed data, mean and median coincide.

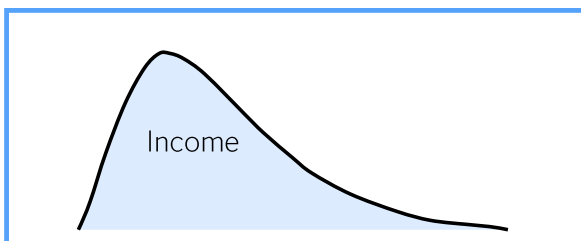
In the case of a high number of outliers in your data set, it is recommended to use the median instead of the mean. Therefore, median imputation would be used for **Income** as it is positively skewed.

Answer: Mean or Median Imputation

Age and **Income** are two variables that have missing values. You can use either mean or median imputation, but each only once. Variable distributions are given below. Which one you would use and why?



Because the **Age** variable is nearly normal, **mean imputation** would work. Nonetheless, for normally distributed data, **mean and median coincide**.



In the case of a **high number of outliers** in your data set, it is recommended to **use the median** instead of the mean. Therefore, **median imputation** would be used for **Income** as it is positively skewed.

Because the **Age** variable is nearly normal, mean imputation would work. Nonetheless, for a normally distributed data, mean and median coincide.

In the case of a high number of outliers in your data set, it is recommended to use the median instead of the mean. Therefore, median imputation would be used for **Income** as it is positively skewed.

Question: Missing Stock Exchange Price

If in the stock exchange a minimum price has been reached and the exchange process has been stopped, the missing stock exchange price can be replaced with which of the following?

- a. Mean
- b. Median
- c. Minimum
- d. Maximum

Answer: Missing Stock Exchange Price

If in the stock exchange a minimum price has been reached and the exchange process has been stopped, the missing stock exchange price can be replaced with which of the following?

- a. Mean
- b. Median
- ☒ c. Minimum
- d. Maximum

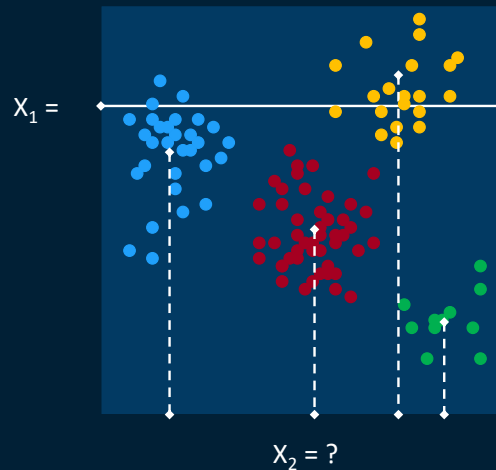
ANSWER: c

If you know that the data must fit a given range, and if the measuring system stops recording and the signal saturates beyond one of such boundaries, you can use either minimum or maximum as the replacement value for missing values. The missing stock exchange price can be replaced with the minimum value of the law's exchange boundary.

Cluster Imputation

Cluster-Mean Imputation:

1. Cluster the cases into relatively homogenous groups.
2. Conduct mean-imputation within each group.
3. For new cases with multiple missing values, use the cluster mean that is closest in all the nonmissing dimensions.



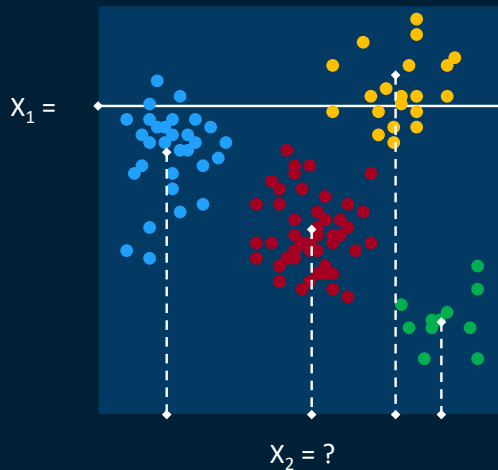
Mean imputation can be refined by using the mean within homogenous groups of the data. Cluster-mean imputation is a somewhat more practical alternative, which is a three-step process.

Step 1: Cluster the cases into relatively homogenous groups.

Step 2: Conduct mean-imputation within each group.

Step 3: For new cases with multiple missing values, use the cluster mean that is closest in all the nonmissing dimensions.

Cluster Imputation



Cluster-Mean Imputation:

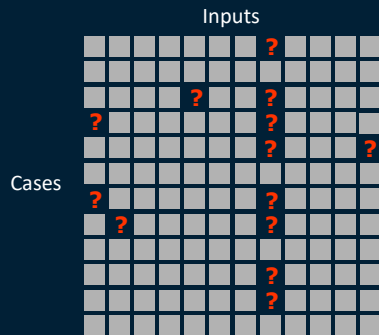
- Can accommodate missingness that depends on the other input variables.
- Different values are imputed for different clusters.
- Imputed values are the same or fixed within each cluster.

This method can accommodate missingness that depends on the other input variables. Different values are imputed for different clusters. However, the imputed values are the same or fixed within each cluster.

Tailored-Value Imputation



provide customized imputations for each case with missing values

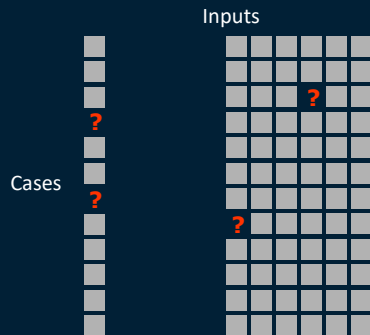


Many times, we might want to avoid the one-size-fits-all approach and provide customized imputations for each case with missing values. This approach is called tailored-value imputation.

Tailored-Value Imputation

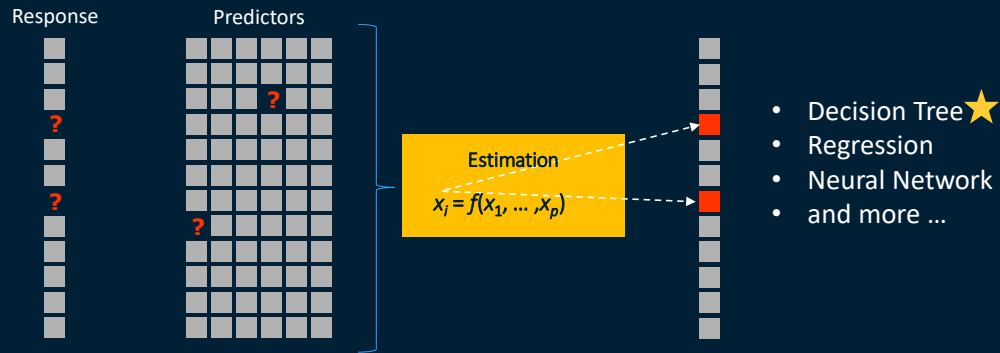


provide customized imputations for each case with missing values



Again, consider the variable in the first column. The remaining inputs will be used in this imputation method, if not all. For example, out of 11 remaining inputs, six might be used based on some criteria.

Tailored-Value Imputation



Mean imputation uses the unconditional mean of the variable. Similarly, cluster-mean imputation uses the unconditional mean of the variable within each segment. An attractive extension would be to use the mean conditional on the other inputs, that is, viewing the missing value problem as a prediction problem. When an input's value is unknown, you can use this model to predict or estimate the unknown missing value. This is referred to as tailored-value imputation. Specifically, k models could be built, one for each input variable using the other inputs as predictors. This would presumably give better imputations and be able to accommodate missingness that depends on the values of the other inputs. However, a complication of tailored-value imputation is that the other inputs might themselves have missing values. Consequently, the k imputation models also need to accommodate missing values. For decision trees, missing values are not problematic. Hence, decision trees are the preferred models for this purpose.

Question: Imputation Methods

If you need to accommodate missingness that depends on the other input variables, which of the following imputation methods you can use? (Select all that apply)

- a. Fixed-value imputation
- b. Tailored-value imputation
- c. Cluster imputation

Answer: Imputation Methods

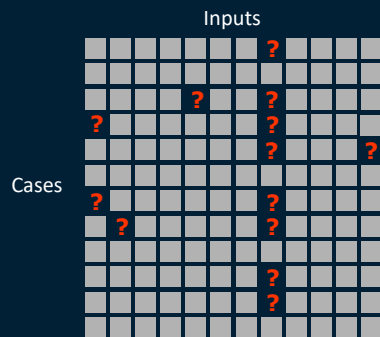
If you need to accommodate missingness that depends on the other input variables, which of the following imputation methods you can use? (Select all that apply)

- a. Fixed-value imputation
- ☒ b. Tailored-value imputation
- ☒ c. Cluster imputation

ANSWER: b and c

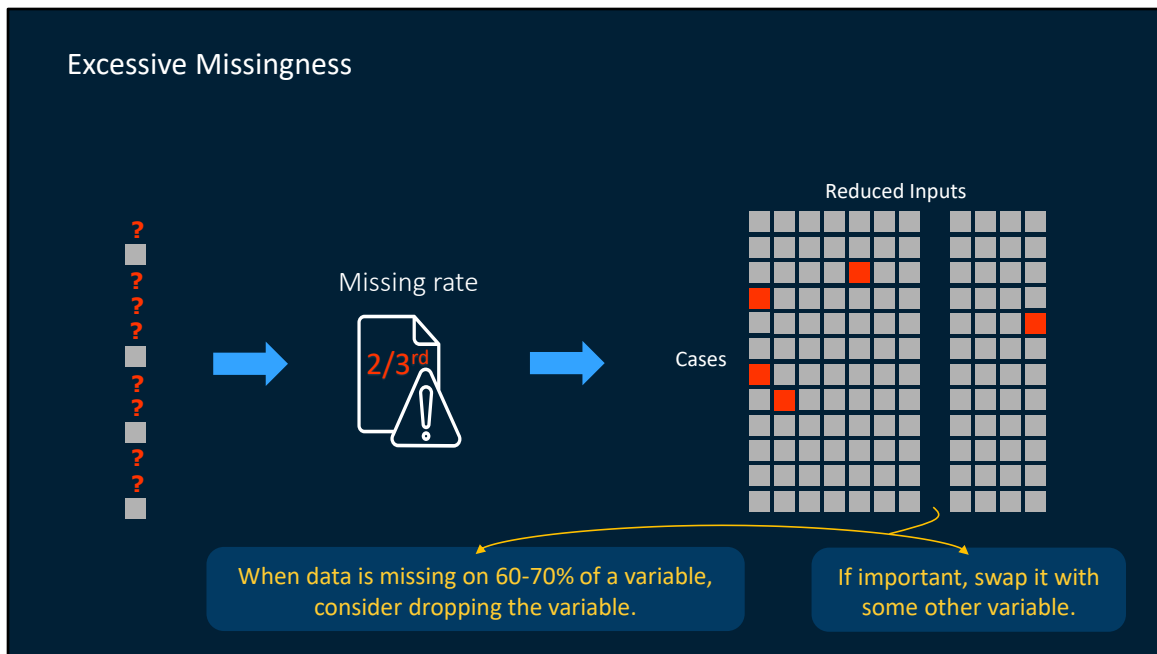
In cluster imputation, clusters are created using other input variables and then using customized imputations within each cluster. Tailored-value imputation accommodates more customization by providing estimated imputations for each case with missing values using other input variables. However, fixed-value imputation does not consider other input variables at all.

Excessive Missingness



There are situations when a variable has a lot of missing values. Let's consider the variable that has the most missing values in this table.

Excessive Missingness



Two-thirds of the values are missing in this variable. The variable can be dropped from the analysis due to its vulnerability of inducing high bias in your analysis. It is certainly not logical to get 67 percent missing values imputed based on 33 percent of the available values. As a general rule, when data is missing on 60 to 70 percent of a variable, you should consider dropping the variable.

If the variable is a very important predictor for the target variable, then swapping the variable with some other variable could be an option.

You must specify the threshold for rejecting inputs with missing values. You generally set this up beforehand while defining the metadata. An input is rejected if it has a higher percentage of missing values than the specified maximum percent. The default threshold of missing values in almost all SAS solutions is 50 percent.

Missing Value Indicators

Incomplete
Data

34
63
.
22
26
54
18
.
47
20

Median = 30

Completed
Data

34
63
30
22
26
54
18
30
49
20

Missing
Indicator

0
0
1
0
0
0
0
1
0
0

Simply **knowing** whether the original variable was missing can **improve the predictive capabilities** of the model!



If a very large percentage of values is missing, then the variable might be better handled by omitting it from the analysis or by creating the binary missing indicator only. The missing indicator is 1 when the corresponding input is missing and 0 otherwise. Each missing indicator becomes an input to the model. Cases often contain missing values for a reason. If the reason for the missing value is in some way related to the target variable, useful predictive information is lost. It might seem strange, but sometimes just knowledge of missingness - simply knowing whether the original variable was missing or not - can improve the predictive capabilities of the model.

This strategy is somewhat unsophisticated and yet satisfies two of the most important considerations in predictive modeling: capturing the relationship of missingness with the target and efficient scorability. A new case is easily scored. First, replace the missing values with any valid imputed value like the median from the development data, and then apply the prediction model.

If a very small percentage of the values is missing (less than 0.1 percent), then the missing indicator is probably of little value.

Demo: Managing Missing Values

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Missing Value Imputation

If the mean of an input variable in training data is 35 and the mean for the same variable in validation data is 39, which one would you use for missing value imputation? Select all that apply.

- a. 35 for training data and 39 for validation data
- b. 35 for both training and validation data
- c. 39 for both training and validation data
- d. 35 for scoring data

b and d are correct

The validation data is prepared exactly the same way as the training data to prevent information leakage. Validation data represents the unseen scoring data. In statistics and machine learning, leakage is the use of information in the model training process, which would not be expected to be available at prediction time, causing the predictive scores to overestimate the model's utility when run in a production environment.

Making the missing value imputation process part of the modeling process allays the modified distribution concern. Any modifications made to the training data are also made to the validation data and the remainder of the modeling population.

Answer: Missing Value Imputation

If the mean of an input variable in training data is 35 and the mean for the same variable in validation data is 39, which one would you use for missing value imputation? Select all that apply.

- a. 35 for training data and 39 for validation data
- ☒ b. 35 for both training and validation data
- c. 39 for both training and validation data
- ☒ d. 35 for scoring data



b and d are correct

The validation data is prepared exactly the same way as the training data to prevent information leakage. Validation data represents the unseen scoring data. In statistics and machine learning, leakage is the use of information in the model training process, which would not be expected to be available at prediction time, causing the predictive scores to overestimate the model's utility when run in a production environment.

Making the missing value imputation process part of the modeling process allays the modified distribution concern. Any modifications made to the training data are also made to the validation data and the remainder of the modeling population.

Question: Outliers

Outliers are anomalous data values. Are there any outliers here?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

Answer: Outliers

Outliers are anomalous data values. Are there any outliers here?

	C1	C2	C3	C4	C5	C6	C7
	cking	#cking	ADB	NSF	dirdep	SVG	bal
R1	Y	1	468.11	1	1876	Y	1208
R2	Y	1	68.75	0	0	Y	0
R3	Y	1	212.04	0	6		0
R4		.	.	0	0	Y	4301
R5	y	2	585.05	0	7218	Y	234
R6	Y	1	47.69	2	1256		238
R7		1	4687.7	0	0		0
R8	Y	.	.	1	0	Y	1208
R9		.	.	.	1598		0
R10		1	0.00	0	0		0
R11	Y	3	89981.12	0	0	Y	45662
R12	Y	2	585.05	0	7218	Y	234

What Is an Outlier?

- An outlier is a data point that is noticeably different from the rest.

Variable: Fish size



- An outlier can easily be detected using
 - visualization techniques: box plot, scatter plot...
 - mathematical functions: IQR, Z-scores...
 - consultation with a domain expert.

In statistics, an *outlier* is an observation point that is distant from other observations. That means an outlier is something that is an odd-one-out or the one that is different from the crowd. For example, if you have a variable as simple as fish size, as illustrated here, the biggest fish seems to be an outlier as it is so much bigger than all the other fish.

There is no one method to detect outliers. However, you can detect them using visualization techniques such as box plots and scatter plots, or with mathematical functions such as IQR and Z-scores. To determine whether a value is an outlier, you must consult a domain expert.

Outliers and Machine Learning Models

- Outliers might not be problematic for all machine learning models.
- Traditional models like linear and logistic regression are easily influenced by outliers resulting in less precise models.
- Outliers also present a problem for some of the activation functions used in neural networks.
- For some other machine learning models, outliers can result in prolonged training times.
- Outliers are not always awkward. At times, detecting outliers is the primary analysis objective, like detecting fraud (anomaly detection).

Outliers are generally concerning. However, they might not be problematic for all machine learning models. Some of the machine learning models are at risk to the statistics and distribution of the input variables. Modeling accuracy generally decreases as the percentage of outliers and their magnitude increases.

Regression models are sensitive to extreme or outlying values in the input space. Inputs with highly skewed or highly kurtotic distributions can be selected over inputs that yield better overall predictions. How do you score cases with unusual values? Regression models make their best predictions for cases near the centers of the input distributions. If an input can have (on rare occasion) extreme or *outlying* values, the regression model might not respond appropriately.

Models like neural networks are also affected by extreme or unusual values. This is because some activation functions that do not squeeze the output under certain range (squashing property) result in huge accuracy losses in the presence of outliers.

In supervised models, outliers can mislead the training process resulting in elongated training times or leading to the development of less precise models.

It's not that outliers are always awkward or indicative of a problem. Outliers sometimes can be helpful indicators. For example, in some applications of data analytics like fraud detection. Anomaly detection as it is sometime called becomes important because here, the exception rather than the rule might be of interest. Support vector data description (SVDD) is a common machine learning technique for detecting anomalies.

Dealing with Outliers

- Do nothing

Variable: Fish size



Outliers are data points that differ from the general trend of the data by more than is expected. An outlier might be an erroneous data point, or one that is atypical compared with the rest of the data.

If there's a chance that the outlier will not significantly alter the outcome, you might accept it as is.

Dealing with Outliers

- Do nothing
- Delete outliers

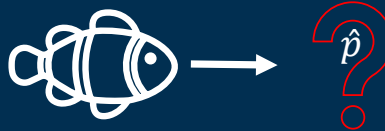
suitable only for wrong data points

results in a biased model

Training data



Scoring data

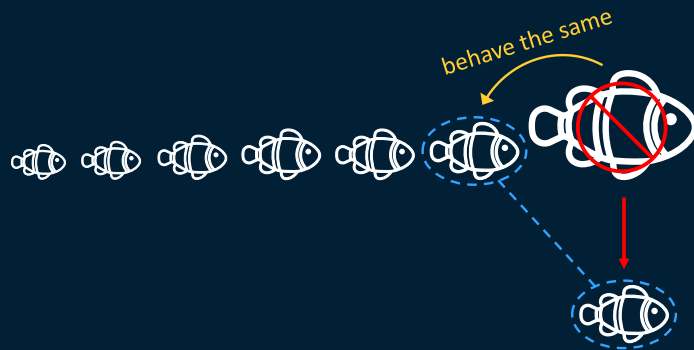


It's easy to completely remove outliers from your data set to stop them from skewing your analysis. However, this temptation is sometimes the wrong choice. You should reserve deletion only for data points that are definitely wrong. For example, you discover that the equipment in the lab was miscalibrated.

Just deleting an outlier from the training data results in a biased model. For example, the model won't be applicable for the largest fish, if it got deleted. What happens when this type of largest fish comes up in the scoring data? Certainly, the generated predicted scores might not be accurate for such cases. Remember, deleting outliers should be the last resort, not the first.

Dealing with Outliers

- Do nothing
- Delete outliers
- **Cap outliers**

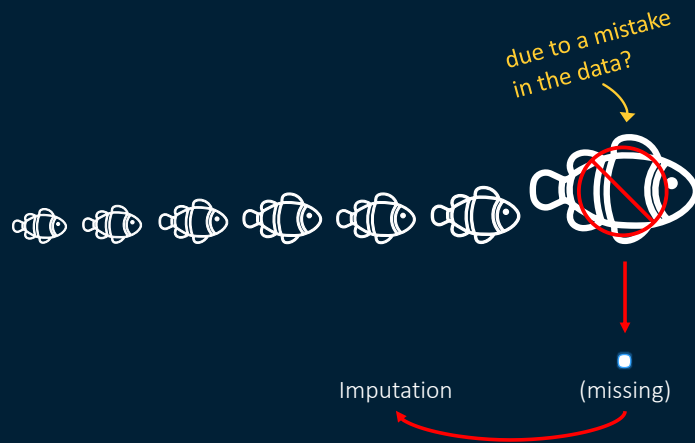


Another way to handle true outliers is to cap them. For example, if you're using age, you might find that people above a certain age behave in the same way as those with a lower age. In this case, you can cap the age value at a level that keeps that intact.

Let's see how capping outliers would look in our fish example. If the largest fish, which is an outlier, behaves like the second-largest fish, you can cap the fish size. Treat all big fish beyond a certain size just like smaller fish.

Dealing with Outliers

- Do nothing
- Delete outliers
- Cap outliers
- Assign new value(s)



What if an outlier seems to be due to a mistake in your data? In that case, treat it like a missing value, and then try imputing a new value. Common imputation methods include using the mean of a variable or utilizing an estimation model to predict the missing value.

Dealing with Outliers

- Do nothing
- Delete outliers
- Cap outliers
- Assign a new value
- **Bin variable**

Bucket binning

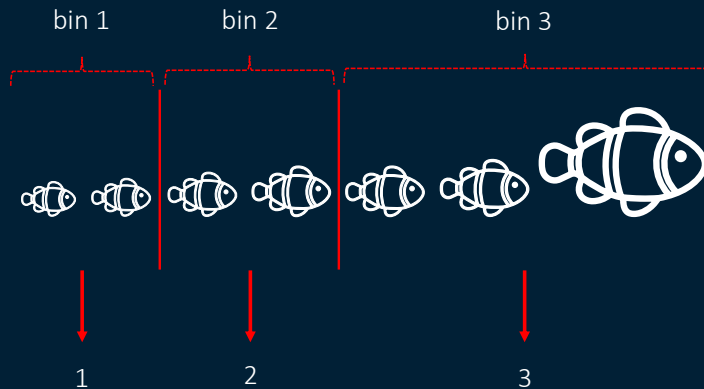


equal interval
width of each
bin

Quantile binning



equal number
of observations
in each bin



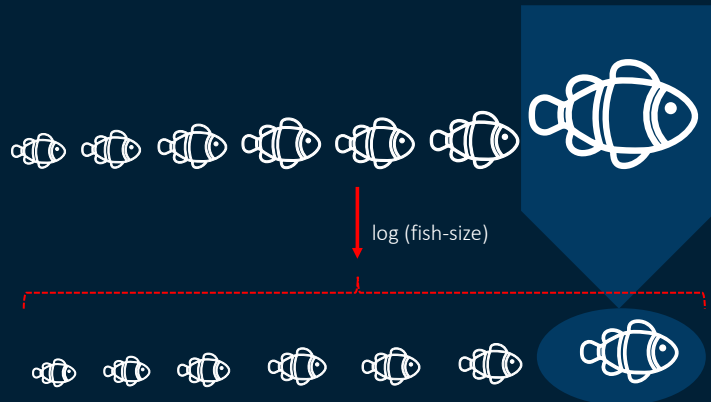
Binning is another effective method for reducing the influence of outliers. The original data values are divided into small intervals known as bins and then they are replaced by a general value calculated for that bin (for example, ranks).

Binning is also useful for classifying missing values, generating variables for detecting nonlinear effects, and reducing categorical variables with a high number of levels to something more manageable.

Equal frequency binning (quantile binning) and equal width binning (bucket binning) are two common binning methods. Bucketing divides observations into equal-length buckets. Within a variable, each bin has the same interval. Quantile binning bins observations into approximately equal-sized bins using the pseudo-quantiles.

Dealing with Outliers

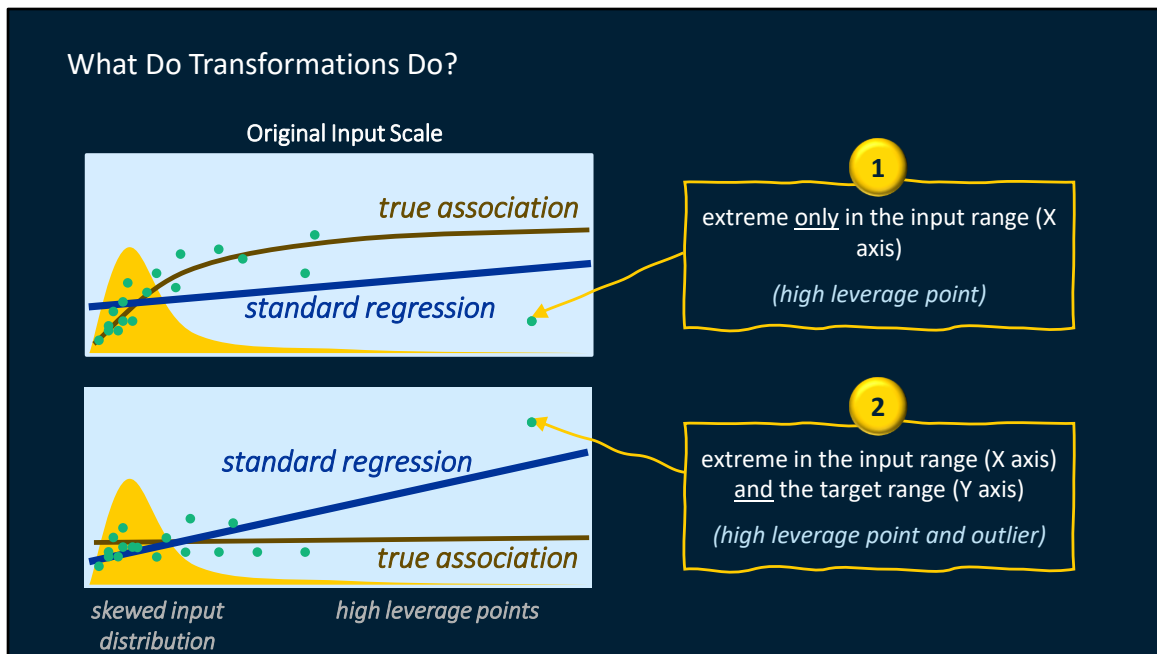
- Do nothing
- Delete outliers
- Cap outliers
- Assign a new value
- Bin variable
- **Transform variable**



A different approach to true outliers could be to try creating a transformation of the data rather than using the data itself. For example, try creating a logarithmic of fish size and working with that new field instead. The influence of the largest fish is then mitigated to a great extent using an appropriate transformation.

When data are skewed or there are outliers, transformation of the data might correct the problem. But how? We'll discuss that next.

What Do Transformations Do?



Let's consider classical regression analysis that makes no assumptions about the distribution of inputs. The only assumption is that the expected value of the target is a linear combination of input measurements. Then, why should you worry about extreme input distributions?

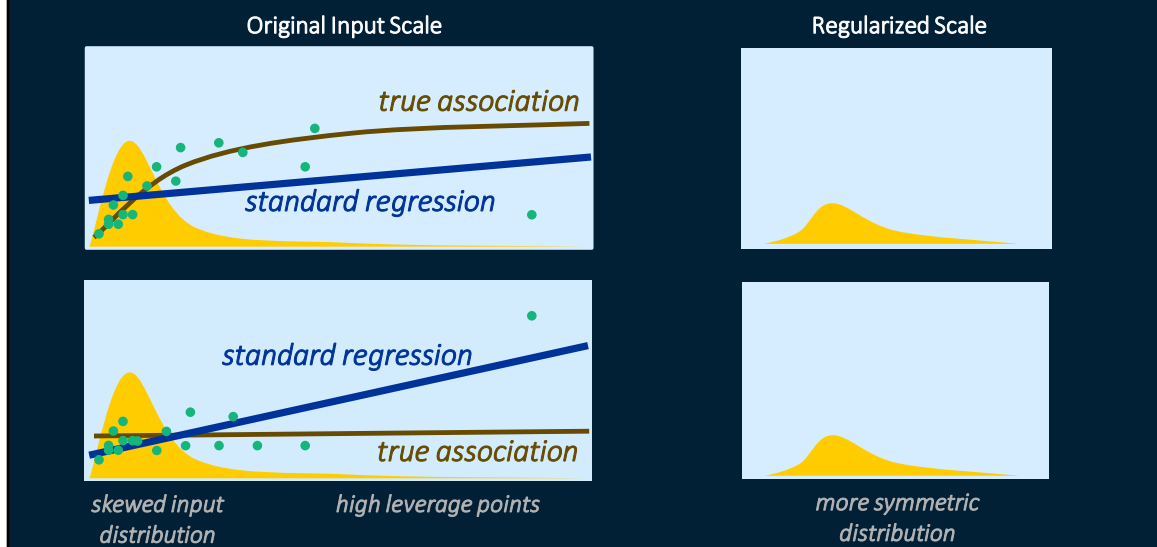
There are at least two compelling reasons.

First, in most real-world applications, the relationship between expected target value and input value does not increase without bound. Rather, it typically tapers off to some horizontal asymptote. Standard regression models are unable to accommodate such a relationship. Notice that the extreme observation is a "high leverage" point (only on the input axis) and not an "outlier" (on the target axis).

Second, as a point expands from the overall mean of a distribution, the point has more influence, on model fit. Models built on inputs with extreme distributions attempt to optimize fit for the most extreme points at the cost of fit for the bulk of the data, usually near the input mean. This can result in an exaggeration or an understating of an input's association with the target. Notice that the extreme observation is an "outlier" (on the target axis) as well as a "high leverage" point (on the input axis).

Note that the first concern can be addressed by abandoning standard regression models for more flexible machine learning models. Abandoning standard regression models is often done at the cost of model interpretability, and more importantly, failure to address the second concern of leverage occurs.

What Do Transformations Do?



A simpler, and arguably more effective, approach transforms or regularizes offending inputs in order to eliminate extreme values.

What Do Transformations Do?

Original Input Scale



Regularized Scale



Then, a standard regression model can be accurately fit using the transformed input in place of the original input.

What Do Transformations Do?

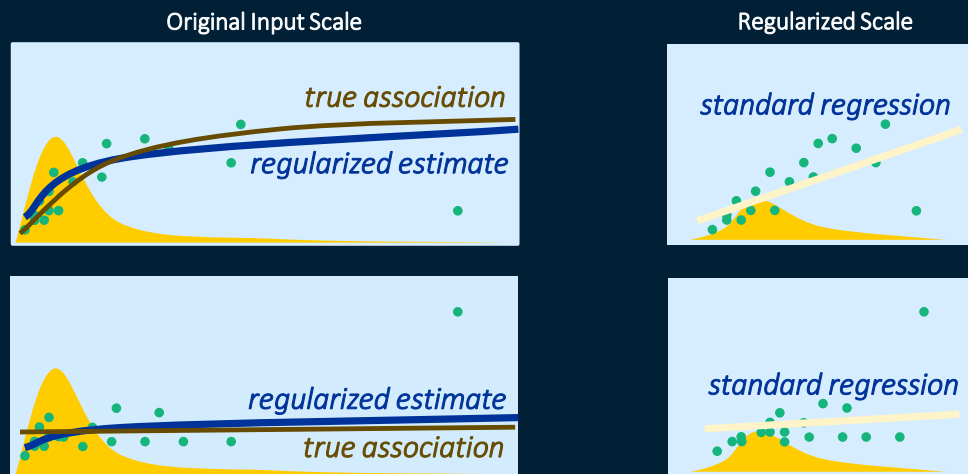
Original Input Scale



Regularized Scale



What Do Transformations Do?



Often this can solve both problems mentioned previously. This not only mitigates the influence of extreme cases, but also creates the desired asymptotic association between input and target on the original input scale. The benefit of this approach is improved model performance. The cost, of course, is increased difficulty in model interpretation.

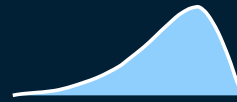
Simple Transformations

Right-skewed data



- Logarithmic $\log(x)$ ← most common
- Root $\sqrt[n]{x}$ ← weakest
- Reciprocal $1/x$ ← strongest

Left-skewed data



can be made stronger with higher power

- Square x^2
- Exponential e^x

can be made stronger with higher base

Which transformation should I use?



You might be wondering, *which transformation should I use?* Well, this is a million-dollar question, but like other analytical issues, this too does not have a definitive answer.

When data is right-skewed or positively skewed, logarithmic is the most commonly used transformation. The strength of this transformation can be somewhat altered by the base of the logarithm. It cannot be used on negative numbers or 0.

Another transformation is taking the root of the variable. This is the weakest transformation. If your transformation of choice is too strong, you will end up with data skewed in the other direction. The strongest transformation is taking the reciprocal of the variable. This transformation is stronger with higher exponents.

If your data is left-skewed or negatively skewed, you can use square and exponential transformations. Square transformation can be made stronger with higher power. Similarly, exponential transformation will become stronger with higher base.

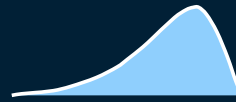
Simple Transformations

Right-skewed data



- Logarithmic $\log(x)$
- Root $\sqrt[n]{x}$
- Reciprocal $1/x$

Left-skewed data



- Square x^2
- Exponential e^x

Light-tailed data

platykurtic
(-kurtosis)



$\text{abs}(x - \text{median})$

Heavy-tailed data

leptokurtic
(+kurtosis)



Deviations of the tail from normality are usually less critical than skewness and might not need transformation after all. However, what if you still want to transform light- and heavy-tailed data? How would you do this?

First, subtract the data points from the median to set your data to a median of 0. Then use an appropriate transformation for skewed data on the absolute deviations from 0 on either side. For heavy-tailed data, use transformations for right skewed data to pull in on the median. For light-tailed data, use transformations for left-skewed data to push data away from the median.

Note that categorical inputs also warrant appropriate transformations. However, this is not discussed in this course.

Question: Variable Transformations

Which statement below is True about transformations of input variables in a regression analysis?

- a. They are never a good idea.
- b. They help model assumptions match the assumptions of maximum likelihood estimation.
- c. They are performed to reduce the bias in model predictions.
- d. They are typically done on nominal (categorical) inputs.

Answer: Variable Transformations

Which statement below is True about transformations of input variables in a regression analysis?

- a. They are never a good idea.
- b. They help model assumptions match the assumptions of maximum likelihood estimation.
- ☒ c. They are performed to reduce the bias in model predictions.
- d. They are typically done on nominal (categorical) inputs.

Answer: (c)

Transformations are useful when you want to improve the fit of a model to the data. For example, transformations can be used to stabilize variances, remove nonlinearity, improve additivity, and correct nonnormality in variables, thereby reducing the bias in model predictions.

Demo: Transforming Inputs

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Problems with Too Many Variables

Data Difficulties and Modeling Issues

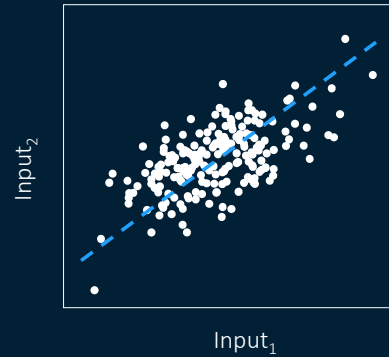


- Visualizing big data
- Processing noisy data
- ➔ • **Handling high dimensional data**
- Dealing with distinct variability in inputs
- Using available data judiciously
- Adjusting model flexibility and complexity
- Understanding learning and tuning of models
- Making models explainable

There is a natural tendency to think that the more features that you have to describe the observations in your data, the more information you have about those observations. Thus, the more accurate your models should be. However, that is true only to a point because having more columns is not always helpful.

Problems with Too Many Variables

- Redundancy
(correlated inputs)

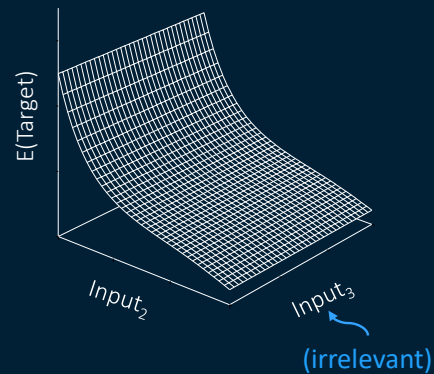


When the number of potential input variables is large, some of these inputs can be redundant. Redundant inputs complicate model construction. One consequence of input redundancy might be model instability.

Here *Input1* and *Input2* are correlated and thus redundant. Knowing the value of *Input1* gives you a good idea of the value of *Input2*. Generally, there would not be any gain in having both into the model. Redundancy is discussed in a previous lesson.

Problems with Too Many Variables

- Redundancy
- Irrelevancy
(no association with target)



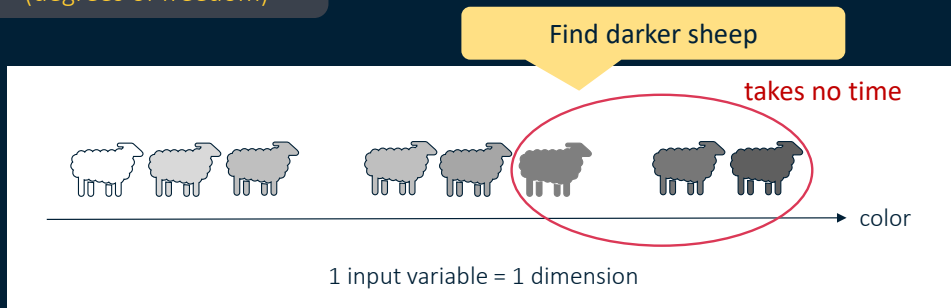
Irrelevancy creeps in when you have many input variables. An irrelevant input does not provide information about the target and therefore does not contribute to the model. Adding irrelevant features unnecessarily complicates and increases the expense of training models.

Here, predictions change with Input_2 but much less with input Input_3 , thus, Input_3 is irrelevant. There would not be any gain in adding Input_3 into the model. Irrelevancy is discussed in a previous lesson.

Problems with Too Many Variables

- Redundancy
- Irrelevancy
- **Curse of Dimensionality (degrees of freedom)**

- number of features can exceed the number of observations
- calculations become extremely difficult



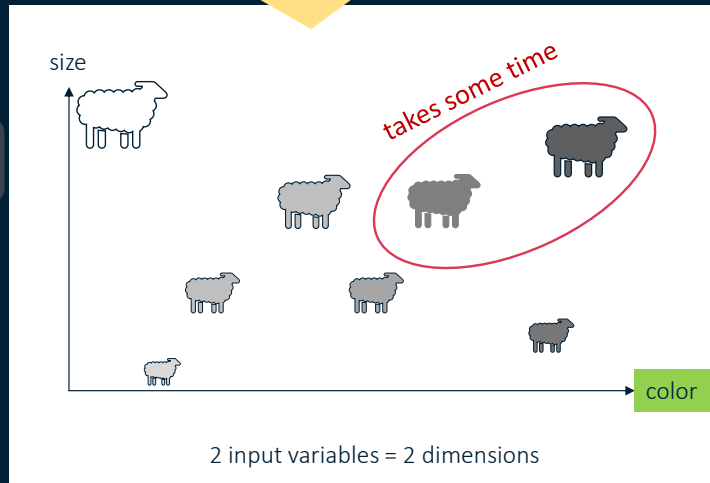
The *dimension* refers to the number of input variables (actually input degrees of freedom). We generally encounter large numbers of input variables in machine learning. The number of variables often has a greater effect on computational performance than the number of cases. Curse of dimensionality means that the number of dimensions is staggeringly high, so high that calculations become extremely difficult, and the number of features can exceed the number of observations.

Consider a simple example of finding darker sheep. The only available feature is color of sheep. It's pretty easy for you to select perhaps the three most gray sheep. Here, data density is very high. Hence, you will not face any difficulty to trace these sheep.

Problems with Too Many Variables

- Redundancy
- Irrelevancy
- **Curse of Dimensionality (degrees of freedom)**

Find darker and bigger sheep



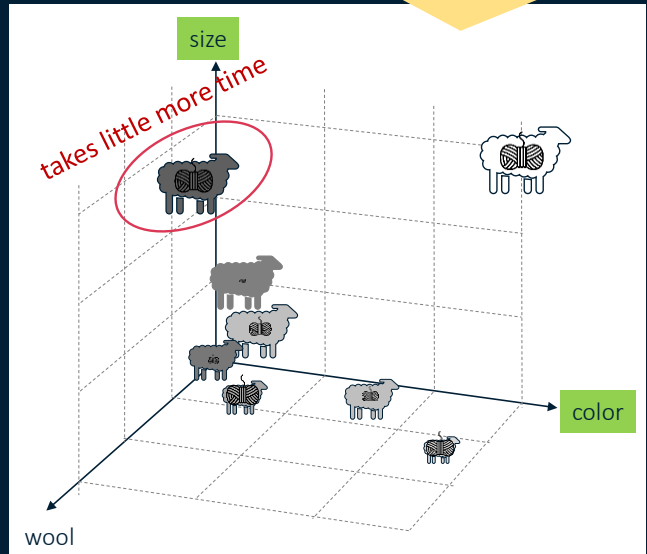
If you add another feature such as size of sheep to its color, then the complexity of a data set increases. If you need to find the darker and bigger sheep, you will take a little more time as compared to previous scenario. Perhaps you to select two sheep in this case. Thus, high dimensionality limits the ability to explore and model the relationships among the variables.

Problems with Too Many Variables

Find darker, bigger sheep with high wool density

- Redundancy
- Irrelevancy
- Curse of Dimensionality (degrees of freedom)

3 input variables = 3 dimensions



If you add one more feature, wool density to its size and color, the complexity of a data set increases rapidly with increasing dimensionality (Breiman et al. 1984). If you need to find the darker, bigger sheep with high wool density, you will take a little more time as compared to previous scenario. Perhaps you to select one sheep in this case. This added dimensionality further limits your ability to explore and model the relationships among the variables.

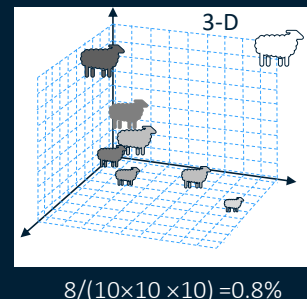
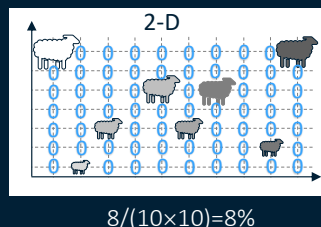
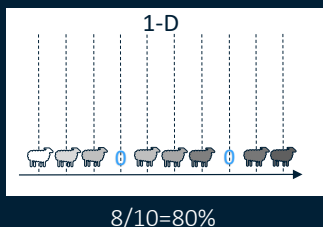
Problems with Too Many Variables

- Redundancy
- Irrelevancy
- Curse of Dimensionality

- **Sparseness**
(many zeros in computation matrix)

Common in applied machine learning:

- count data
- data encodings that map categories to counts
- natural language processing



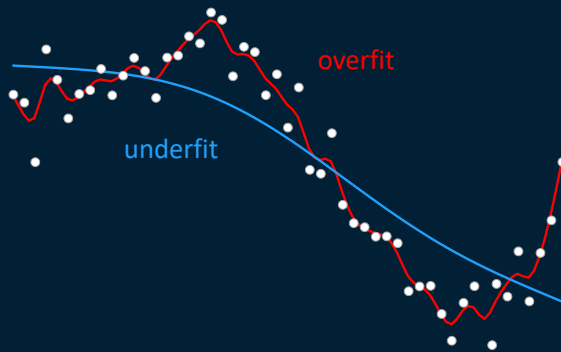
Sparseness addresses the so-called *curse of dimensionality*, which tells us that as you add features, you are introducing greater sparsity to your observations in that feature space. Sparseness refers to a matrix of numbers that includes many zeros or values that will not significantly impact a calculation.

In this simple example, where in one dimension you have the space well covered with eight data points that covers 80% of the space, when we add another dimension, those same eight data points cover a much smaller percentage of the feature space, only 8%. This sparsity grows exponentially with each feature added. The practical meaning of sparseness is that there is much space between the data points in the high-dimensional space built up by all the used input variables. The data points have a long way to the neighbors—that is, they are lonely. Many variables consist of one or two values (for example, 0 or 1). This is a source of problems for many machine learning methods as such matrices are computationally expensive.

Large sparse matrices are common in applied machine learning, such as in data that contains counts, data encodings that map categories to counts, and even in natural language processing. The solution to representing and working with sparse matrices is to use an alternate data structure to represent the sparse data.

Problems with Too Many Variables

- Redundancy
- Irrelevancy
- Curse of Dimensionality
- Sparseness
- **Overfitting**
(model not generalize well)

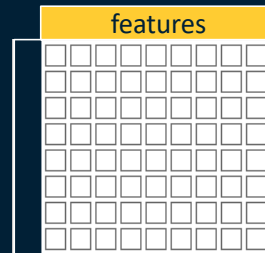


Overfitting occurs when the model fits the training data too well but cannot generalize to unseen data sets. Overfitting was discussed in the previous lesson. Overfitting will be discussed in a broader sense in the next section.

Problems with Too Many Variables

- Redundancy
- Irrelevancy
- Curse of Dimensionality
- Sparseness
- Overfitting

*Feature engineering
is the solution.*



Find and create the best inputs to use in modeling.
~ simpler models, shorter training times, improved accuracy ~

A critical part of the success of a machine learning project is producing a good set of features to train on. You need to include features to characterize the observations, but it is important to be selective about the features that you choose to incorporate in your modeling process. For example, with the curse of dimensionality, the amount of data that you would need to collect to get a meaningful predictive model would be enormous. Therefore, methods are needed to transform the data from this high-dimensional space to fewer dimensions. Feature engineering is the process of determining the best set of inputs to use in building predictive models.

In the context of the entire analytics life cycle, feature engineering is typically done as part of the data preprocessing leading into modeling. It is a big part of the 80% project time that you hear about being spent in preparation for modeling. And it is a very important step that can make a big difference in the accuracy and effectiveness of your models.

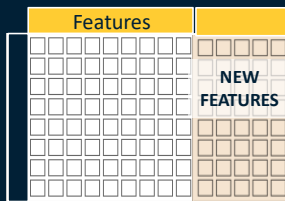
You need great features that describe the structures inherent in your data. The reduced dimensionality of the data set can result in simple, more accurate models that train in less time.

Types of Feature Engineering

Feature
Construction

Feature
Selection

Feature
Extraction



Generally, feature engineering falls into three main categories: construction, selection, and extraction of features.

In machine learning applications, the variables relevant to the analysis rarely come prefabricated. Therefore, they must be created. This is not so much an application of analytical techniques as it is a manual assessment of the existing features, ensuring a clear understanding of the problem being addressed, and then augmenting the data set by aggregating existing features into fewer, and possibly more representative, features or even decomposing individual features into multiple features that better represent the problem.

Types of Feature Engineering



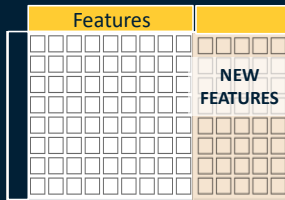
Feature Construction Example

Claim Date	Accident Date-Time	derived features		
		Delay	Season	Dark
11NOV22	102322/12:38	19	fall	0
22DEC21	012321/01:42	333	winter	1
26APR21	042321/03:05	3	spring	1
02JUL20	070220/06:25	0	summer	0
08MAR22	123021/18:33	69	winter	0
15DEC22	061222/18:12	1816	summer	0
09NOV20	110520/22:14	4	fall	1

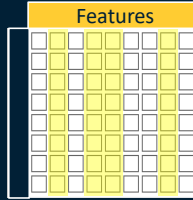
For example, the date on which an auto accident took place and an insurance claim was filed might not be useful predictors of fraud. Derived variables such **Delay**, the time between the two events, might be more useful. Similarly, **Season** and **Dark** (whether it was dark at the time of accident) would contribute more to checking the legitimacy of the insurance claim.

Types of Feature Engineering

Feature Construction



Feature Selection



Feature Extraction

Feature engineering is essentially referring to applying techniques to reduce the feature set. Feature selection, that is, reducing the number of inputs, is the obvious way to thwart the curse of dimensionality. Unfortunately, reducing the dimension is also an easy way to disregard important information. The two principal reasons for eliminating a variable are redundancy and irrelevancy. A redundant input does not give any new information that was not already explained by other inputs. An irrelevant feature does not provide information about the target. Various criteria can be used to filter the set of features down to the most important ones.

Feature Selection Example

Response to Direct Mail	Household Income	Home Value
1	76,000	370,000
0	21,000	102,000
0	50,000	243,000
0	67,000	325,600
1	100,000	485,800
0	45,000	146,500
1	28,000	202,700

Illustrated here is a simple example of a direct marketing campaign where Response to Direct Mail is the binary target. In this example, Household Income and Home Value are obviously redundant features. Knowing Household Income gives you a good idea of Home Value. Thus, only one of them should be selected.

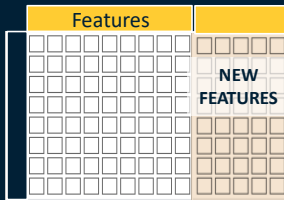
Feature Selection Example

Response to Direct Mail	Household Income	Home Value	Shoe Size	Previous Solicits
1	76,000	370,000	10	3
0	21,000	102,000	11	1
0	50,000	243,000	9	0
0	67,000	325,600	7	1
1	100,000	485,800	6	4
0	45,000	146,500	13	0
1	28,000	202,700	12	2
↑ irrelevant ↑				
↑ relevant ↑				

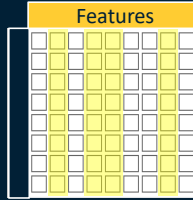
In addition, predictions change with the Previous Solicits feature, but not with Shoe Size. Shoe Size does not provide information about the target. So, Previous Solicits should be selected.

Types of Feature Engineering

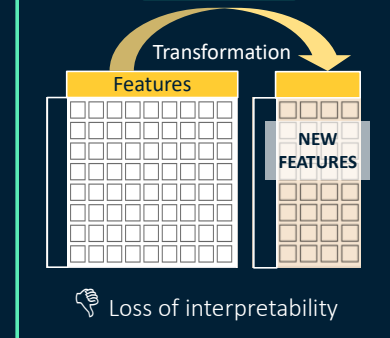
Feature Construction



Feature Selection



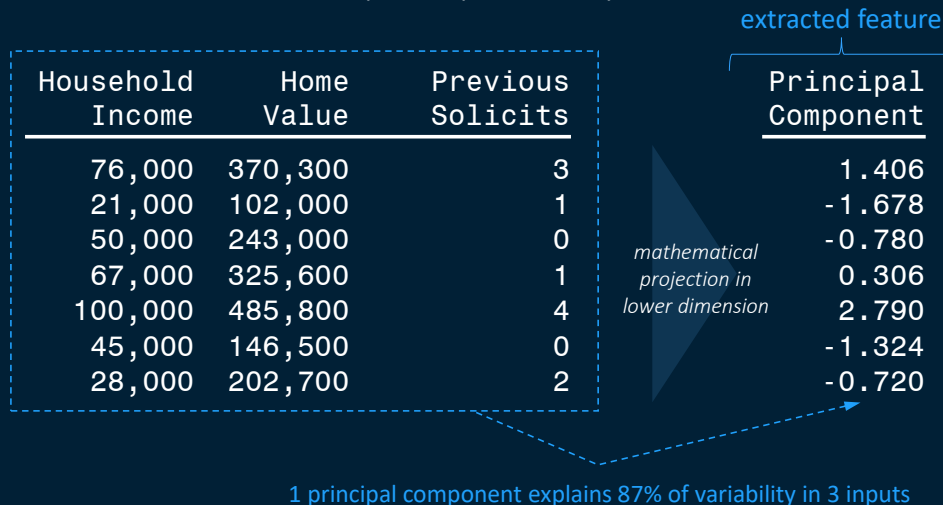
Feature Extraction



Feature extraction is another technique to reduce the feature set. Feature extraction transforms the data in a way such that fewer columns can be used to represent most of the information contained in the original features. Feature extraction derives new features, which leads to lower-dimensional representation of the feature information. With feature extraction techniques, the newly derived features are not interpretable in the context of the original problem. However, this is very often accepted as simply another layer of the model, representing the input to output relationships, and the ultimate benefits realized can be significant.

Feature Extraction Example

Principal Component Analysis (PCA)



Principal component analysis (PCA) is a common feature extraction method in machine learning. Practically, PCA converts an original matrix of features into a new data set of fewer features. PCA creates as many new features (called the *principal components*) as the number of input variables. However, you often select only the first few principal components to represent your data. That is, it reduces the number of features by constructing a new, smaller number variables that capture a significant portion of the information found in the original features. Here, three features are represented by the first principal component, and it explains 87% of the variability of the original input space.

Singular value decomposition and *autoencoders* are some other feature extraction methods.

Question: Feature Engineering

The key difference between 'selection' and 'extraction' is that feature selection keeps a subset of the original features and feature extraction creates brand new ones.

- a. True
- b. False

Answer: Feature Engineering

The key difference between 'selection' and 'extraction' is that feature selection keeps a subset of the original features and feature extraction creates brand new ones.

- ☒ a. True
- ☐ b. False

Answer: True

Some variable reduction methods (like decision trees, regressions) use the original features as inputs into subsequent models (feature selection). Others (like principal component analysis) use combinations of the original features as inputs into subsequent models (feature extraction).

Demo: Performing Feature Selection

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Distinctly Scaled Variables

Data Difficulties and Modeling Issues

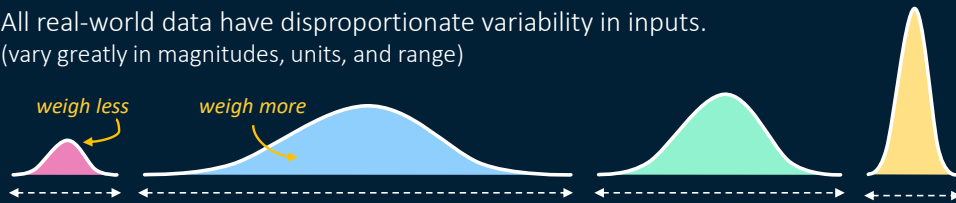


- Visualizing big data
- Processing noisy data
- Handling high dimensional data
- ➔ • Dealing with distinct variability in inputs
- Using available data judiciously
- Adjusting model flexibility and complexity
- Understanding learning and tuning of models
- Making models explainable

Next, we'll continue our discussion of data difficulties. Let's address how to deal with distinctly scaled variables.

Distinctly Scaled Variables

- All real-world data have disproportionate variability in inputs. (vary greatly in magnitudes, units, and range)



- When the features of a data set differ greatly in scale, models that are sensitive to the scale of the inputs would be affected:
 - slower model training
 - imprecise parameter estimates
 - arbitrary weight to predictors
 - model instability

Ensuring that features are within a similar scale is imperative. However, most of the time your data set will contain features that vary greatly in magnitudes, units, and range. Many machine learning algorithms are sensitive when the data is not scaled evenly. It might result in slower training times, imprecise estimates of parameters, and unstable models. The features with high magnitudes will weigh in a lot more in the distance calculations than features with low magnitudes. To suppress this effect, we need to bring all features to the same level of magnitude. This can be achieved by *scaling*.

Feature Scaling

How can you analyze construction milestones of these statues?

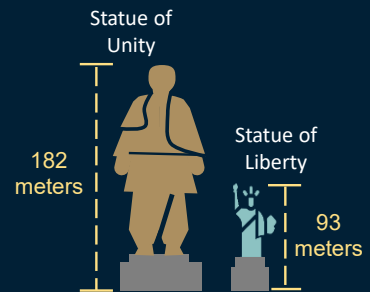


amount of steel and concrete used



construction time

Problem: Using variables without standardization can give variables with larger ranges greater importance in the analysis.



Scaling

Solution: Rescale the data to comparable scales.



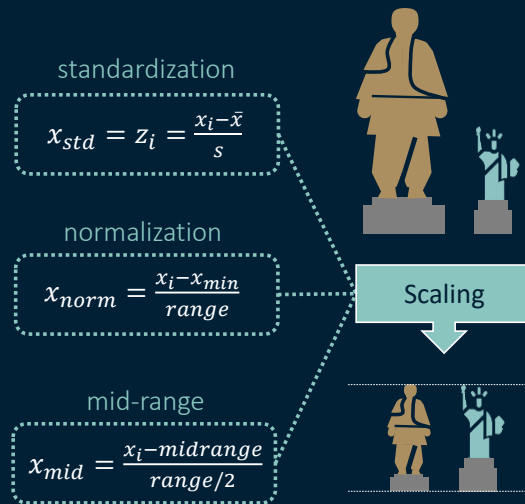
Imagine you are tasked with analyzing construction milestones of the Statue of Unity in India with the Statue of Liberty in the United States. You just need to remember that the statues are not of the same height. The Statue of Unity is 182 meters tall whereas the Statue of Liberty is 93 meters. You cannot compare their construction-related milestones such as construction time or amount of steel and concrete used unless you make them similar in height.

Model stability and parameter estimate precision are influenced during multivariate analysis when multi-scaled variables are used. For example, in boundary detection, a variable that ranges between 0 and 100 will outweigh a variable that ranges between 0 and 1. Using variables without standardization can give variables with larger ranges greater importance in the analysis. Rescaling the data to comparable scales can prevent this problem.

Feature Scaling

Common rescaling methods:

- Standardization
(mostly -3 to 3)
- Normalization
(0 to 1)
- Mid-range
(-1 to 1)



There are many rescaling methods available. Listed here are the ones commonly used in machine learning.

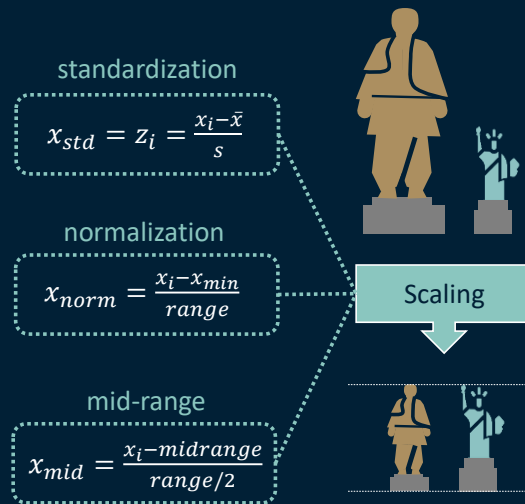
Standardization is the method of rescaling your data to have a mean of zero and a standard deviation of one. A standardized variable is sometimes called a Z-score or a standard score. Here's the equation to standardize your data, where x_i is the original value, $\bar{x}$ is the variable's mean, and s is the variable's standard deviation. The absolute value of z_i represents the distance between the raw score and the population mean in units of the standard deviation. z_i is negative when the original value is below the mean, positive when above. z_i is between negative 3 and 3 for most data.

Normalization is another rescaling method with many meanings in statistics and statistical applications. Most commonly, normalizing rescales numeric data between 0 and 1 using the equation shown here, where x_{min} is the variable's minimum value, and range is the difference between variable's maximum value and minimum value.

Normalizing, or range standardization, is the basis for another type of standardization known as *midrange*. Midrange is calculated as $(x_{max} + x_{min})/2$. Midrange scaling transforms a variable so that its range is -1 to 1 and its midrange value is 0.

Feature Scaling

- ✓ Gradient descent-based algorithms
Neural network , Regression *
- ✓ Distance-based algorithms
KNN, K-means, SVM
- ✗ Tree-based algorithms
Decision Tree, Forest, Gradient Boosting



Some machine learning algorithms are sensitive to feature scaling and others are virtually invariant to it.

Machine learning algorithms like neural network - and, at times, linear regression, logistic regression, and so on, that use gradient descent as an optimization technique - require data to be scaled. When performing regression analysis, standardizing multi-scale variables can help reduce multicollinearity issues for models containing interaction terms. Standardizing continuous predictor variables in a neural network is extremely important. Having features on a similar scale can help the gradient descent converge more quickly toward the minima.

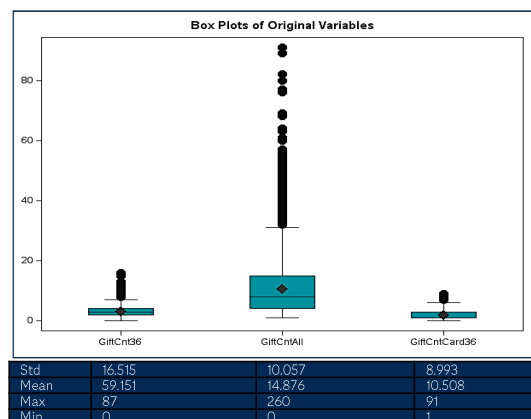
*Note that if you want to run a linear or logistic regression on big data that cannot fit into memory, the coefficients can be optimized using gradient descent.

Distance algorithms like K-nearest neighbor, K-means, and support vector machines are most affected by the range of features. This is because behind the scenes, they are using distances between data points to determine their similarity. Therefore, we scale our data before employing a distance-based algorithm so that all the features contribute equally to the result. Standardizing or normalizing data is also important in principal component analysis (PCA) because it projects your original data onto orthogonal directions, which maximize the variance.

However, tree-based analyses are not sensitive to outliers and do not require variable transformations. As a result, standardization of multi-scaled data is not necessary for decision tree, random forest, or gradient boosting algorithms.

Think About It: Feature Scaling Method

Suppose you are tasked with creating a neural network model on the **PVA_DONORS** data using the three variables shown below.



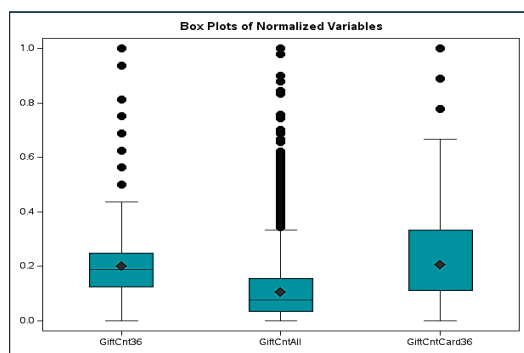
There is no obvious answer to this question. It really depends on the data and the intended model. In neural networks algorithms that require data on a 0 to 1 scale, normalization is an essential pre-processing step. Normalization is good to use when the distribution of data do not follow a Gaussian distribution. It can be useful in algorithms that do not assume any distribution of the data like *k*-nearest neighbors.

On the other hand, standardization can be helpful in cases where the data follow a Gaussian distribution. Though this does not have to be necessarily true. Standardization does not have a bounding range, so, even if there are outliers in the data, they will not be affected by standardization.

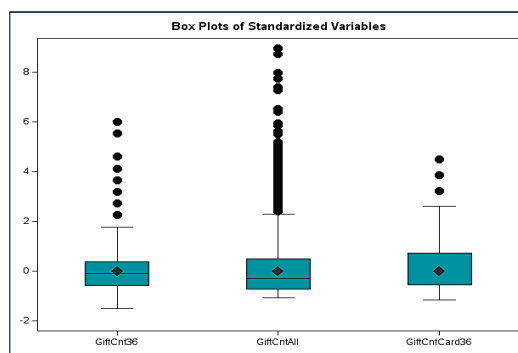
Think About It: Feature Scaling Method

Which feature scaling method would you use to correct disproportionate scales, if any?

Normalization



Standardization



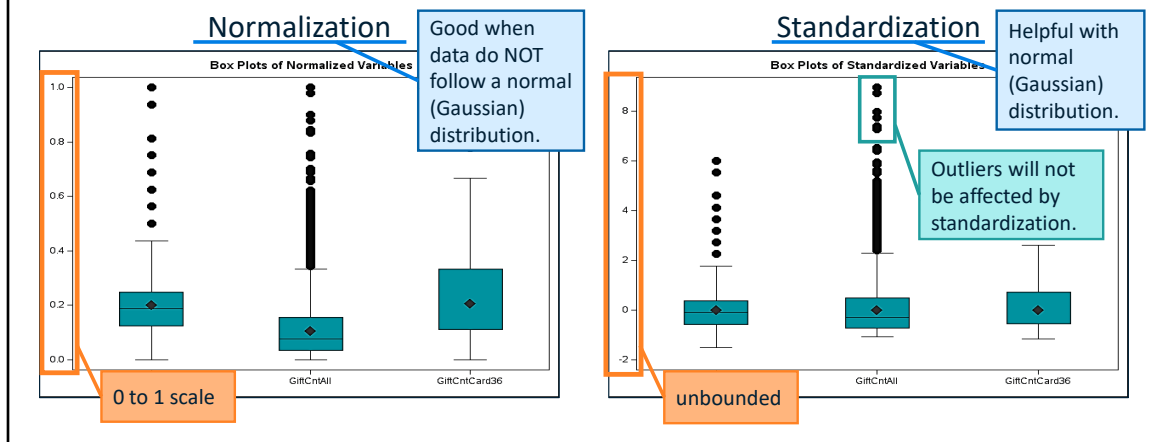
There is no obvious answer to this question. It really depends on the data and the intended model. In neural networks algorithms that require data on a 0 to 1 scale, normalization is an essential pre-processing step. Normalization is good to use when the distribution of data do not follow a Gaussian distribution. It can be useful in algorithms that do not assume any distribution of the data like *k*-nearest neighbors.

On the other hand, standardization can be helpful in cases where the data follow a Gaussian distribution. Though this does not have to be necessarily true. Standardization does not have a bounding range, so, even if there are outliers in the data, they will not be affected by standardization.

Answer: Feature Scaling Method



Which feature scaling method would you use to correct disproportionate scales, if any? *It depends on the data!*



There is no obvious answer to this question. It really depends on the data and the intended model. In neural networks algorithms that require data on a 0 to 1 scale, normalization is an essential pre-processing step. Normalization is good to use when the distribution of data do not follow a Gaussian distribution. It can be useful in algorithms that do not assume any distribution of the data like k -nearest neighbors.

On the other hand, standardization can be helpful in cases where the data follow a Gaussian distribution. Though this does not have to be necessarily true. Standardization does not have a bounding range, so, even if there are outliers in the data, they will not be affected by standardization.

Question: Feature Scaling

Why do some of machine learning algorithms require you to scale a set of features in your data?

- a. to make features more interpretable
- b. to make your data values positive
- c. because it's fun to do
- d. because many algorithms are sensitive to unevenly scaled data

Answer: Feature Scaling

Why do some of machine learning algorithms require you to scale a set of features in your data?

- a. to make features more interpretable
- b. to make your data values positive
- c. because it's fun to do
- ☒ d. because many algorithms are sensitive to unevenly scaled data

Answer: d

Many machine learning algorithms are sensitive when the data are not scaled evenly. The features with high magnitudes will weigh in a lot more in the distance calculations than features with low magnitudes. To suppress this effect, we need to bring all features to the same level of magnitude. This can be achieved by *scaling*.

5.3 Model Evaluation, Estimation, and Post-training Tasks



Modeling Challenges with Machine Learning Data

Data Difficulties and Modeling Issues



- Visualizing big data
- Processing noisy data
- Handling high dimensional data
- Dealing with distinct variability in inputs
- ➡ • Using available data judiciously
- Adjusting model flexibility and complexity
- Understanding learning and tuning of models
- Making models explainable

Throughout this lesson, we've been discussing data difficulties and modeling issues. Let's discuss another challenge. What if you don't have a lot of relevant data? How can you use the available data judiciously?

Modeling Challenges with Machine Learning Data

- Small data sets



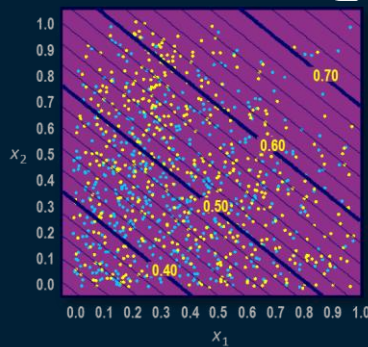
Despite having great inclination toward machine learning, you might not have enough data. Machine learning models are by and large data hungry, and their performance relies heavily on the size of training data available. In many cases, it is difficult to get training data sets that are large enough. For example, you might not have any data available for a newly launched product. Although it is difficult to evaluate the minimum amount of data required, the very nature of your project will influence the amount of data that you will need. For example, training models to identify texts, images, and videos usually require more data.

The real problem with insufficient data lies in the fact that with less data, variance increases. High variability in models means that the model will fit the training data perfectly but will stop working as soon as new data is fed into it.

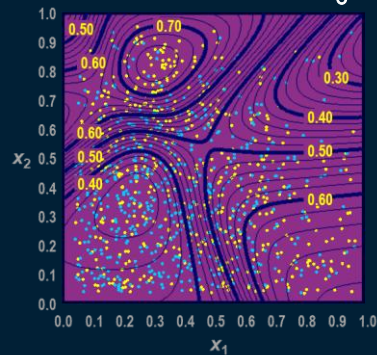
Modeling Challenges with Machine Learning Data

- Small data sets

statistical models or simpler
machine learning models



data-hungry complex
machine learning models



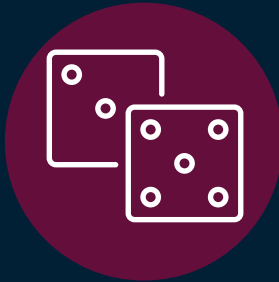
Building traditional statistical models like regressions, or simpler machine learning models like short decision trees, is one of the most common approaches that can help with building predictive models from small data sets.

In general, the simpler the algorithm, the better it will learn from small data sets. From a machine learning perspective, small data requires models that have low complexity (or high bias) to avoid overfitting the model to the data.

More than the size of the data, *relevance* of data is imperative.

Modeling Challenges with Machine Learning Data

- Small data sets
- Lack of relevant data



How much signal do you have in your data?

true underlying pattern from data

$$\text{Target} = \text{Signal} + \text{Noise}$$

Systematic Variation

Random Variation

Explained

Unexplained

Predictable

Unpredictable

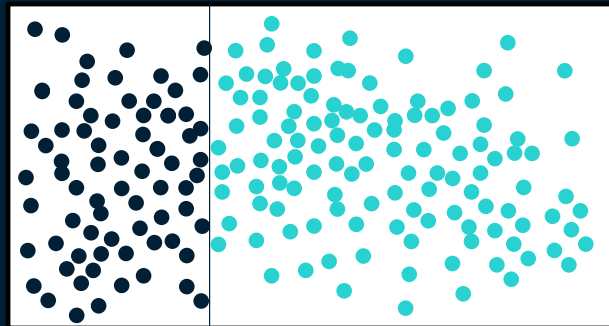
At times you have enough data, but that data might not be sufficiently relevant. Novice modelers who get caught up in the world of exotic algorithms often fall into the trap of thinking the computer can solve any problem given enough time and data. However, consider the problem of predicting the next roll of the dice in a Las Vegas casino. You could have one million data points of historic rolls of the dice, and you would have no better chance of predicting the next roll than if you had no data.

When you build a model, you assume that your data has two parts, signal and noise. You can think of the "signal" as the true underlying pattern that you want to learn from the data. Signal is the repeatable process that you hope to capture and describe. It is the information that you care about. The signal is what lets the model generalize to new situations. "Noise," on the other hand, refers to the irrelevant information or randomness in a data set. The random errors are referred to as noise. Noise is therefore inevitable and unpredictable. They are equivalent to the unexplained part of your statistical model.

A signal-to-noise ratio is a measure of the amount of background noise with respect to the primary input signal.

Modeling Challenges with Machine Learning Data

Pure Separation =
Pure Signal =
No Noise



INPUT

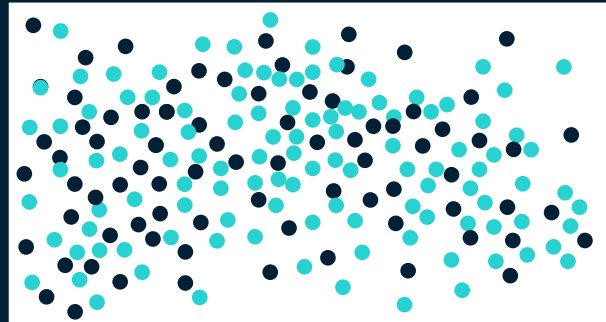
Target: Primary Outcome= ● Secondary Outcome= ●

This graphic illustrates the pure signal situation. In this case, the training data can be perfectly separated into primary or secondary outcomes using a linear decision boundary. You rarely expect to see this in the real world. Unfortunately, some people who are new to predictive modeling are disappointed when the methods do not perfectly categorize with this type of accuracy.

Modeling Challenges with Machine Learning Data

No Separation =
Pure Noise =
No Signal

INPUT₁



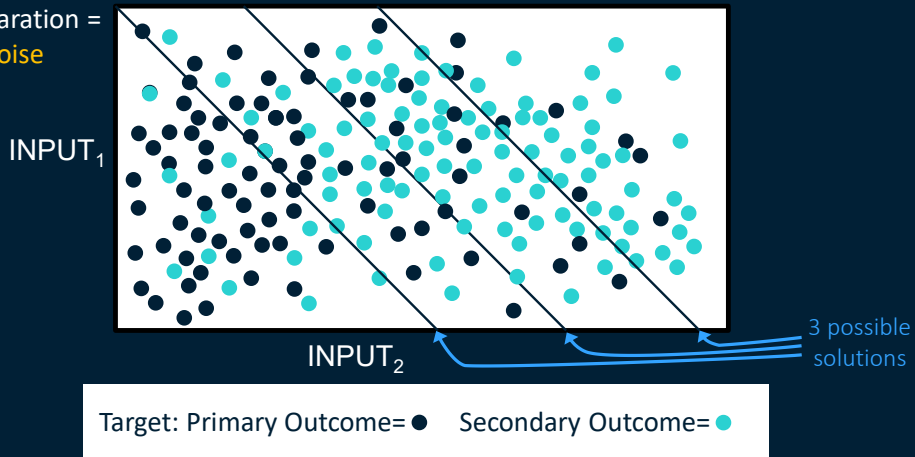
INPUT₂

Target: Primary Outcome=● Secondary Outcome=●

At the other extreme is the pure noise situation. In this case, the training data appears to have no patterns upon which to base a model that can separate the primary outcomes from the secondary outcomes. This situation is more common than you might like. Although pure signal is very rare, *pure noise* can actually occur in reality.

Modeling Challenges with Machine Learning Data

Some Separation =
Signal + Noise

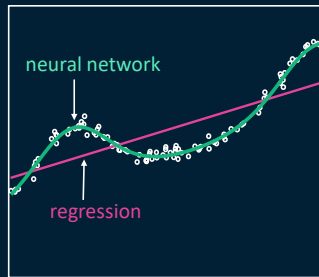


The most common situation in practice is a mixture of signal and noise. You can predict more accurately than randomly guessing. How *well* you predict depends on whether data is dominated by *systematic* variation or *random* variation. The three classifier lines in the plot are meant to represent three different possible solutions.

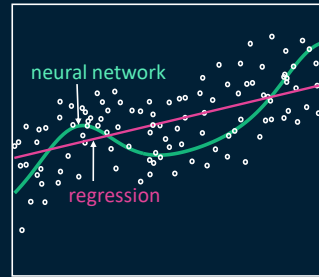
Modeling Challenges with Machine Learning Data

Machine Learning Models Are Not Always Superior!

Noisy data impact models.



$\frac{\text{signal}}{\text{noise}} = \text{high}$



$\frac{\text{signal}}{\text{noise}} = \text{low}$

Sometimes machine learning models like neural networks do not outperform simpler and easier to implement models like regression. The reason for this, in some situations, is found in the signal-to-noise ratio. Noisy data impacts models, including machine learning models.

To illustrate, in the left panel of the above diagram, the nonlinear model that is produced by the neural network would clearly produce a superior fit to the data, as compared to the fit produced by the linear regression model. There is a strong pattern (signal) relative to the amount of variation (noise) around the pattern. The signal-to-noise ratio is high.

The situation in the right panel is different. The signal-to-noise ratio is low. The regression and neural network models will likely offer a comparable fit to the data. So, there would be no advantage to using the more complex neural network.

Question: Signal and Noise

The true underlying pattern that you want to learn from the data is called "noise" whereas the irrelevant information or randomness in a data set is called "signal".

- a. True
- b. False

Answer: Signal and Noise

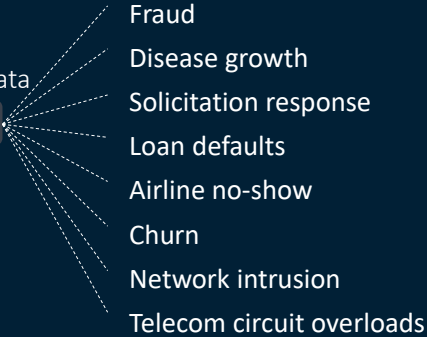
The true underlying pattern that you want to learn from the data is called "noise" whereas the irrelevant information or randomness in a data set is called "signal".

- a. True
- ☒ b. False

Answer: False

The true underlying pattern that you want to learn from the data is called "signal" whereas the irrelevant information or randomness in a data set is called "noise".

More about Modeling Challenges with Machine Learning Data

- Small data sets
 - Lack of relevant data
 - **Rare target event**
- 
- Fraud
 - Disease growth
 - Solicitation response
 - Loan defaults
 - Airline no-show
 - Churn
 - Network intrusion
 - Telecom circuit overloads

In supervised learning, the event of interest is often rare. Thus, in case of a binary target, the event class is in the minority. For example, a utility company might have terabytes of customer usage data and a desire to detect fraud, but it does not know which cases are fraudulent. The data is abundant, but none of it is supervised.

Another example would be health care data where the outcome of interest is progress of some condition across time, but only a tiny fraction of the patients were evaluated at more than one time point.

In direct marketing, if customer history and demographics are available but there is no information about response to a particular solicitation of interest, a test mailing is often used to obtain supervised data.

Loan defaults, airline no-show, churn, computer security threats, electricity, and telecom circuit overloads are some other examples of rare target events.

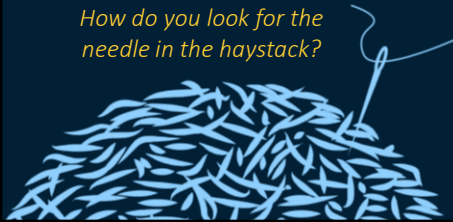
More about Modeling Challenges with Machine Learning Data

- Small data sets
- Lack of relevant data
- **Rare target event**

Problem

- Target search is frequently difficult
- Learns more from the majority group and can easily misclassify the minority event group
- Need a much bigger sample for building a reliable model (Harrell 1997)

*How do you look for the
needle in the haystack?*



The event of interest or the "awkward" level of the target variable is rare, and target search is therefore frequently difficult. It's like looking for a needle in a haystack.

The machine learning algorithm learns more from the majority group and can easily misclassify the small event group.

With rare events, the effective sample size for building a reliable prediction model is closer to three times the number of event cases than to the nominal size of the data set.

In such a scenario, do we have enough cases for modeling?

More about Modeling Challenges with Machine Learning Data

- Small data sets
- Lack of relevant data
- **Rare target event**

How do you look for the needle in the haystack?



Problem

- Target search is frequently difficult
- Learns more from the majority group and can easily misclassify the minority event group
- Need a much bigger sample for building a reliable model (Harrell 1997)

Solution

- Event-based sampling
 - undersampling
 - oversampling
- Use of artificial cases
 - SMOTE
- Unsupervised learning
- Rule induction

There are some different approaches for handling data sets with rare events. Undersampling and oversampling are two forms of event-based sampling that you are already aware of. Event-based sampling was discussed earlier in the course.

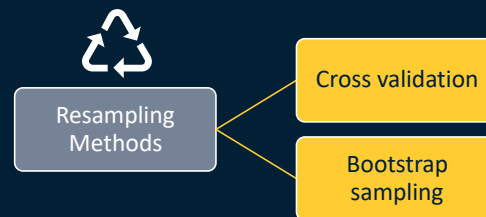
In order to have enough events, creating artificial cases is another approach. Synthetic minority over-sampling technique (SMOTE) is a technique that generate artificial cases. Simply put, SMOTE takes the rare event data points and creates new data points that lie between any two nearest data points joined by a straight line. The algorithm calculates the distance between two data points in the feature space, multiplies the distance by a random number between 0 and 1, and places the new data point at this new distance from one of the data points used for distance calculation.

When the data do not have the capacity to solve the problem, the problem needs to be reformulated. For example, there are unsupervised approaches to detecting anomalous data that might be useful for investigating possible fraud.

There are some specialized machine learning approaches available to improve the classification of rare events, for example, rule induction. Rule induction creates "if-else-then"-type prediction rules from a set of input variables and the target variable. The idea here is to initially generate prediction rules for non-events and then slowly move toward more complex rules that can predict rare events.

More about Modeling Challenges with Machine Learning Data

- Small data sets
- Lack of relevant data
- Rare target event
- Economic use of available data



With moderate sized data sets, data splitting is inefficient. The reduced sample size can severely degrade the fit of the model. Computer-intensive resampling methods have been developed so that all the data can be used for both fitting and honest assessment. Resampling refers to methods for economically using the available data set for robustly assessing model performance and thereby improving model accuracy. Essentially a subset of samples are used to fit a model and the remaining samples are used to estimate the efficacy of the model. This process is repeated multiple times and the results are aggregated and summarized. The differences in techniques usually center around the method in which subsamples are chosen. Although the method sounds overwhelming, the math involved is relatively simple.

Resampling is a set of methods to either estimate the precision of a statistic using *cross validation* or repeat sampling from a given sample known as *bootstrapping*. Thus, the two commonly used resampling methods that you might encounter are cross validation and the bootstrap.

More about Modeling Challenges with Machine Learning Data

- Small data sets
- Lack of relevant data
- Rare target event
- **Economic use of available data**

Cross validation

- used when data sample is small
- splitting the data into multiple portions
- maximizes the availability of training data
- all the data are used for both training and validation purposes
- assessments would be averaged

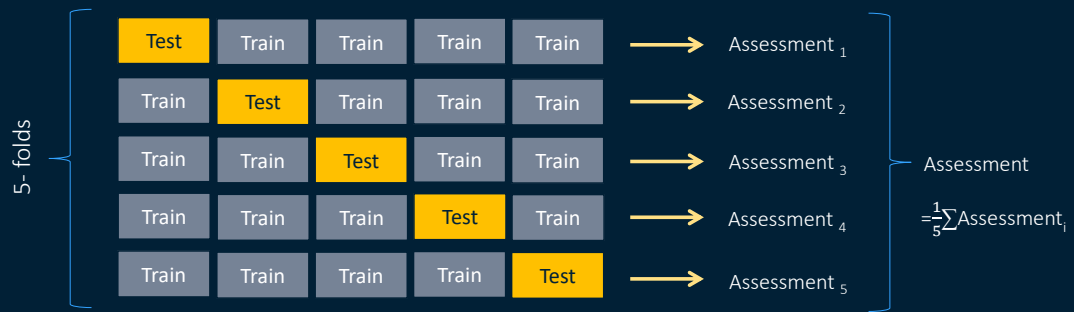
For small and moderate data sets, *k*-fold cross validation (Breiman et al. 1984; Ripley 1996; Hand 1997) is a better strategy.

Partitioning the data into two segments (one for training and one for validation) is a simple but costly technique because a huge portion of data is held out for testing. When the test set is small, the performance measures might be unreliable because of high variability. Fortunately, there is an effective workaround for this issue. For small and moderate data sets, *k*-fold cross validation is a better strategy.

k-fold cross validation is a resampling procedure used to evaluate machine learning models on a limited data sample. Cross validation maximizes the availability of training data by splitting the data into multiple subsets, to use different subsets of data to train and test the model on. The *k*-fold cross validation involves splitting the data into *k* subsets. You take one subset from the bunch and treat it as the test set for the model. And you keep the *k*-1 subset for training the model. You repeat the procedure *k* times, rotating the validation set. *k* is selected based on the size of the data set. This ensures that our model is getting trained and tested on new data at every new step.

Cross Validation Example

k-fold Cross Validation (k=5)



Here is an example of five-fold cross validation. The data would be split into five equal sets. The entire modeling process would be redone on each four-fifths of the data, using the remaining one-fifth for assessment. The five assessments would then be averaged. In this way, all the data are used for both training and validation purposes, which dramatically minimizes the potential error (such as overfitting) found by relying on a fixed split of training and validation data.

Bootstrap Aggregation

- Small data sets
- Lack of relevant data
- Rare target event
- Economic use of available data

Bootstrap
sampling

- large numbers of smaller samples of the same size are repeatedly drawn
- quantify the uncertainty associated with a machine learning model

Bootstrapping is a type of resampling where large numbers of smaller samples of the same size are repeatedly drawn, with replacement, from a single original sample. It is often used as a means of quantifying the uncertainty associated with a machine learning model.

Bootstrap Aggregation

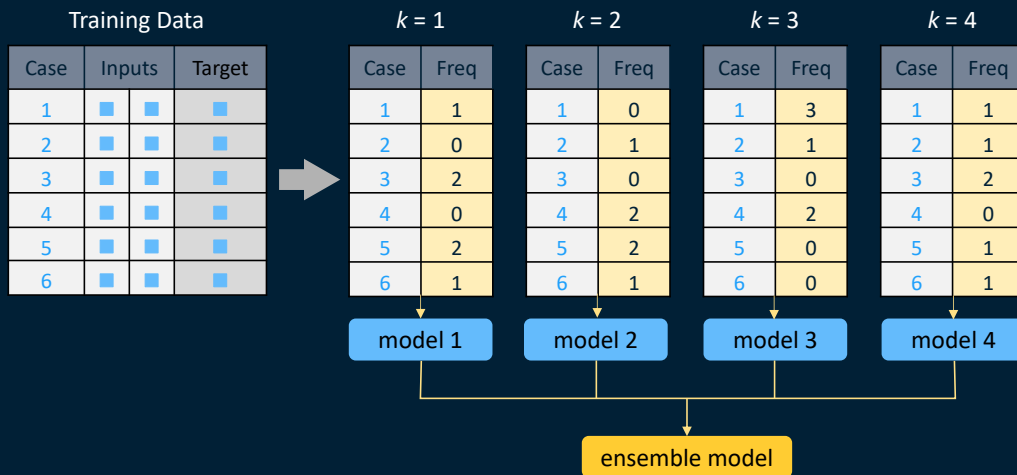
Training Data			$k = 1$		$k = 2$		$k = 3$		$k = 4$	
Case	Inputs	Target	Case	Freq	Case	Freq	Case	Freq	Case	Freq
1	■	■	1	1	1	0	1	3	1	1
2	■	■	2	0	2	1	2	1	2	1
3	■	■	3	2	3	0	3	0	3	2
4	■	■	4	0	4	2	4	2	4	0
5	■	■	5	2	5	2	5	0	5	1
6	■	■	6	1	6	1	6	0	6	1

random with replacement

Let's look at a simple example. From a training data set that has six cases, the first step is to draw four bootstrap samples. Each sample has six cases.

A *bootstrap sample* is a random sample from a training data set. For bagging, a bootstrap sample is drawn with replacement. This means that some of the cases might be left out of the sample, and some cases might be represented more than once. For example, in the first bootstrap sample, cases 3 and 5 appear twice, cases 1 and 6 appear once, and cases 2 and 4 do not appear at all.

Bootstrap Aggregation



After a certain number of equally sized bootstrap samples of a data set are extracted, a machine learning algorithm is applied to each of these samples to get several models. Their predictions are then combined to create an ensemble model. This is commonly known as *bootstrap aggregation* or *bagging*. The idea is to average many noisy but approximately unbiased models to reduce the variance of estimated prediction function.

The ensemble approach relies on combining many relatively weak simple models to obtain a stronger ensemble prediction. This will improve accuracy, reduce bias, reduce variance, and provide robust models in the presence of new data. The commonly observed advantage of the ensemble model is that the combined model is better than the individual models that compose it. It is important to note that the ensemble model can be more accurate than the individual models only if the individual models disagree with one another.

Question: Cross Validation

Cross validation is another method of honest assessment that can be used when data size is small and you do not have a luxury of holding out data separately for validation or testing, or both.

- a. True
- b. False

Answer: Cross Validation

Cross validation is another method of honest assessment that can be used when data size is small and you do not have a luxury of holding out data separately for validation or testing, or both.

- ☒ a. True
- ☐ b. False

Answer: True

Data partitioning is a simple but costly technique because a huge portion of data is held out for validation and testing. Alternatively, k-fold cross validation, which is a resampling procedure, can be used for honest assessment to evaluate machine learning models on a limited data sample.

Model Fitting

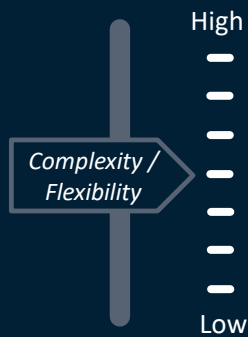
Data Difficulties and Modeling Issues



- Visualizing big data
- Processing noisy data
- Handling high dimensional data
- Dealing with distinct variability in inputs
- Using available data judiciously
- ➔ • **Adjusting model flexibility and complexity**
- Understanding learning and tuning of models
- Making models explainable

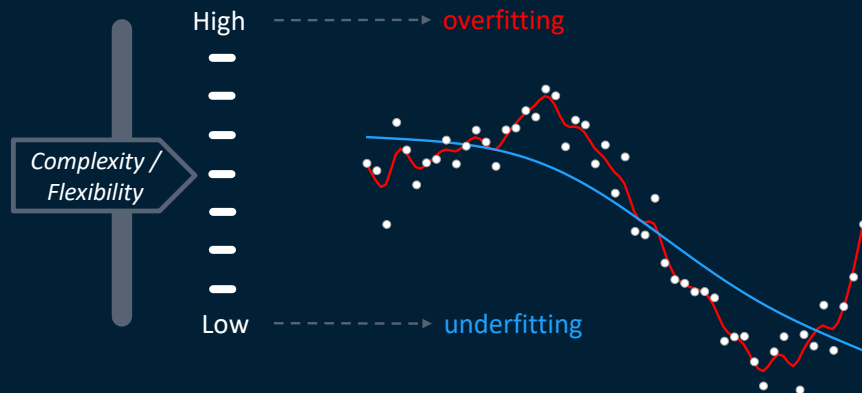
Next, let's talk about challenges with model fitting - how to adjust the *flexibility* and *complexity* of your model.

Model Fitting



In statistics, model fit refers to how well you approximate a target function. This is applicable in machine learning too because supervised machine learning algorithms seek to approximate the unknown mapping function for the target given the input variables. Thus, model fitting refers to how well a machine learning model generalizes to similar data to that on which it was trained. A model that is well-fitted produces more accurate outcomes. Model fitting is closely related to the model complexity, or flexibility, and the related bias-variance trade-off.

Model Fitting

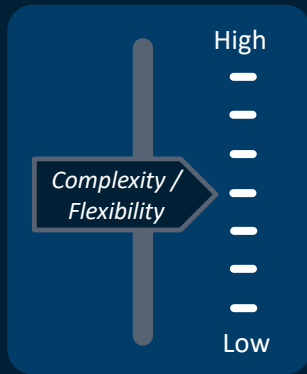


Whether we overfit or underfit can depend on the complexity or flexibility of your model. When the model is too complex, it can start to learn the "noise," or irrelevant information. When the model memorizes the noise and fits too closely to the training set, the model becomes "overfitted," (as shown by the red curve) and it is unable to generalize well to new data. If a model cannot generalize well to new data, then it will not be able to perform the classification or prediction tasks that it was intended for. A model that is overfitted matches the training data too closely. A model that is underfitted doesn't match closely enough. When the model is too simple, or less flexible, the model becomes "underfitted" (as shown in by blue curve).

Here it is important to note that not all machine learning algorithms are overfitted by having only too many inputs. This is especially a problem for models where additional input variables directly mean that additional parameters must be estimated. Examples of such models are regression and neural network models.

Model Fitting

Complexity and Flexibility in Different Models



Model	What Affects Complexity of Each Model
Regressions	# Parameters / Effects
Decision Trees	# Leaves
Gradient Boosting	# Trees
SVMs	# Support vectors
Neural Networks	# Iterations

What precisely constitutes model complexity or flexibility is an intricate matter. Many factors govern whether a model will generalize well. For example, a regression model with more parameters or effects might be considered more complex. A decision tree with more leaves and a gradient boosting model with more trees might be more complex. Similarly, a greater number of support vectors in support vector machines indicates high flexibility. Often with neural networks, we think of a model that takes more training iterations as more complex, and one subject to *early stopping* (or fewer training iterations) as less complex.

Thus, it can be difficult to compare the complexity among members of substantially different model classes. For now, a simple rule of thumb is quite useful: a model that can readily explain arbitrary facts (or noise) is what statisticians view as complex, whereas one that has only a limited expressive power (or signal), and yet still manages to explain the data well, is probably more generalizable and closer to the truth.

Question: Model Complexity

Which of the following statements are True? Select all that apply.

- a. A more complex model generally leads to overfitting.
- b. A more complex model generally leads to underfitting.
- c. Model complexity has nothing to do with overfitting or underfitting.
- d. The less complex the model is, the less flexible it is.

Answer: Model Complexity

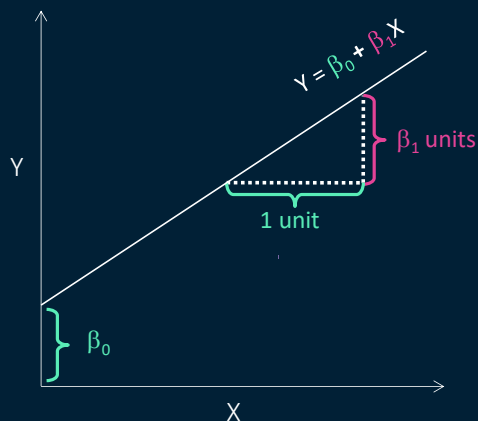
Which of the following statements are True? Select all that apply.

- ☒ a. A more complex model generally leads to overfitting.
- ☐ b. A more complex model generally leads to underfitting.
- ☐ c. Model complexity has nothing to do with overfitting or underfitting.
- ☒ d. The less complex the model is, the less flexible it is.

Correct answer: a and d

Model fitting is closely related to the model complexity, or flexibility, and the related bias-variance trade-off. When the model is too complex, it can start to learn the "noise," or irrelevant information. Whether you overfit or underfit, it can depend on the complexity or flexibility of your model.

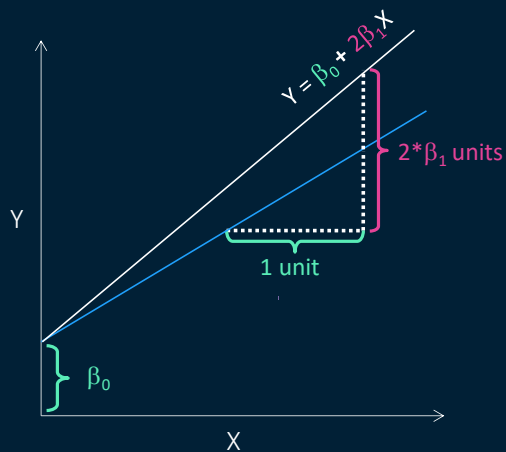
Effect of Magnitude of Coefficients



Many machine learning algorithms do not have a built-in capability of variable selection, and thus they build the models on all inputs supplied. While building a machine learning model, the model can easily be overfitted or underfitted.

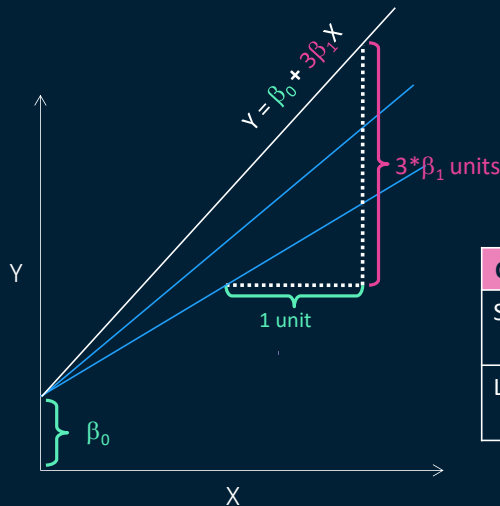
In parametric models, like regression and neural network, the magnitude of coefficients influence the model complexity. To keep things simple, let's consider the case of a simple linear regression. Y is the response variable and X is the predictor variable. In its simplest form, it attempts to fit a straight line to your data set. There are two coefficients: β_0 is the intercept parameter and β_1 is the slope parameter. The equation suggests that for every one unit increase in X, there is β_1 units increase in predicted Y.

Effect of Magnitude of Coefficients



Let's see what happens when the coefficient becomes larger. For example, if the slope gets doubled, the regression line turns out to be steeper. Now the equation suggests that for every one unit increase in X , the prediction for Y increases by two times β_1 units.

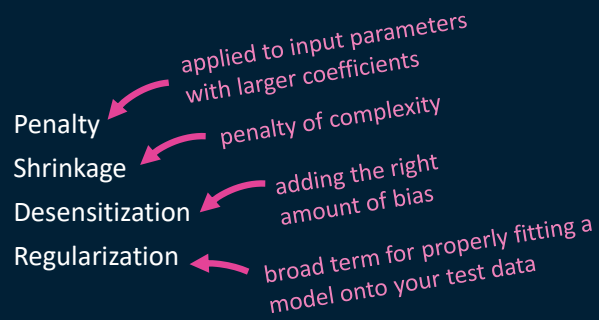
Effect of Magnitude of Coefficients



Coefficients	Result
Small	predictions for Y are less sensitive to changes in X
Large	predictions for Y are more sensitive to changes in X

If the slope gets tripled, the equation suggests that for every one unit increase in X, there is three times β_1 units increase in predicted Y. When the slope of the line is steep, the prediction for Y is very sensitive to relatively small changes in X. When the slope is small, for every one unit increase in X, the prediction for Y barely increases. In other words, when the slope of the line is small, predictions for Y are much less sensitive to changes in X.

Shrinking the Coefficients



To avoid overfitting or underfitting, data scientists and statisticians often talk about penalty, shrinkage, desensitization, or regularization. All these terms refer to a broad range of techniques for artificially forcing your model to be simpler.

We can apply a *penalty* to the input parameters with the larger coefficients, which subsequently limits the amount of variance in the model.

Shrinkage can be thought of as a penalty of complexity. If we set some parameters of the model to exactly zero, then the model is effectively shrunk to have lower dimensionality and be less complex.

Adding the right amount of bias to a model can help make more accurate predictions by *desensitizing* it to some of the noise in the training data.

Regularization is a broader term commonly used in machine learning to properly fit a model onto your validation data. The regularization method will depend on the type of model you're using. For example, you could prune a decision tree, use dropout on a neural network, or add a penalty parameter to the cost function in regression. Many times, the regularization method is a hyperparameter as well, which means it can be tuned through cross validation.

Shrinking the Coefficients



Regularization essentially balances complexity in machine learning models. Regularization refers to techniques that are used to calibrate machine learning models to prevent overfitting or underfitting.

How does regularization penalize or shrink the coefficients?

Shrinking the Coefficients

Regularization Penalty

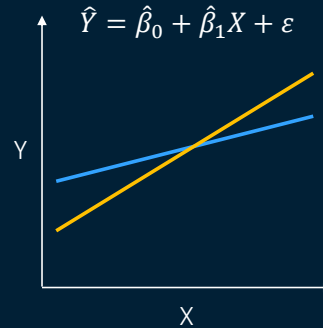
OLS regression minimizes

$$RSS = \sum_{i=1}^n [y_i - (\beta_0 + \beta_1 x_i)]^2$$

Shrinkage/regularization minimizes

$$RSS + \text{Regularization Penalty}$$

- L1 (Lasso) selects a subsample of features
- L2 (Ridge) uses a simpler model
- L12 (Elastic net) both



Recall a simple linear regression model that takes the form shown above the graph on the right. Here, $\hat{Y}$ represents the predicted response, and $\hat{\beta}$ represents the coefficient estimates. The values of these β s are estimated by using the ordinary least squares method that minimizes the sum of the squared residuals.

Regularization is a form of regression that constrains or regularizes the coefficient estimates to shrink them toward zero. In its most simplistic form, regularization adds a penalty to model parameters (all except intercepts), so the model generalizes the data well. Or contrary to OLS, shrinkage or regularization minimizes the penalized sum of squared residuals. Thus, the coefficients are estimated by minimizing a slightly different quantity when you regularize. A regularization penalty is added to the residual sum of squares.

The main idea behind shrinkage methods is to find a new line that doesn't fit the training data as well. So, we introduce a small amount of bias into how the new line is fit to the data. In return for that small amount of bias, we get a significant drop in variance. In other words, by starting with a slightly worse fit, regularization can provide stable predictions in the long run. In our example, we see that the regularization penalty resulted in a line (the blue line) that has a smaller slope, which means that predictions made using this line are less sensitive to the changes in the values of 'X' than the least squares line (the yellow line).

Three types of regularization penalties are often used in machine learning models:

L1 or *Lasso* regularization: L1 uses a shrinkage mechanism to zero out some parameters and selects a subsample of features.

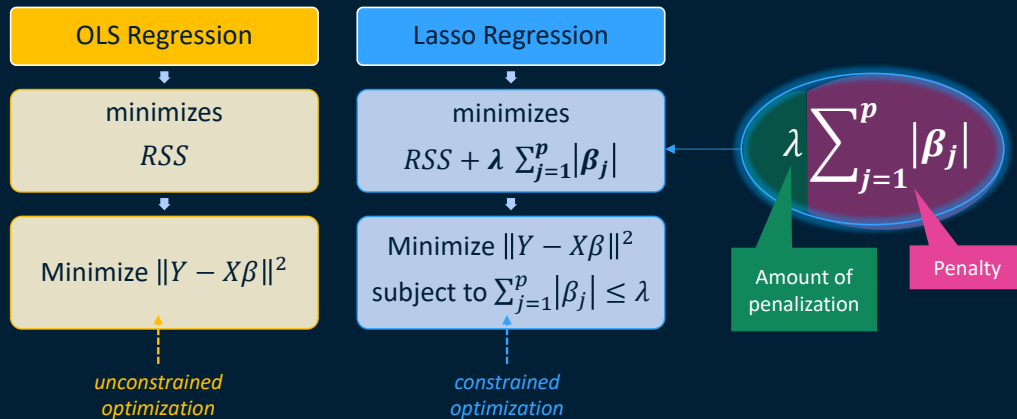
L2 or *Ridge* regularization: L2 adds a penalty on the parameters and uses a simpler model.

L12 or *Elastic net* regularization: L12 is a tradeoff between L2 and L1 regularization and has a penalty that is a mix of L1 and L2 norm. It can perform the function of feature selection while still not discarding too many features.

Lasso, ridge, and elastic net are all part of the linear regression family where the x (input) and y (output) are assumed to have a linear relationship.

Shrinking the Coefficients

L1 Type of Penalty – Lasso Regularization



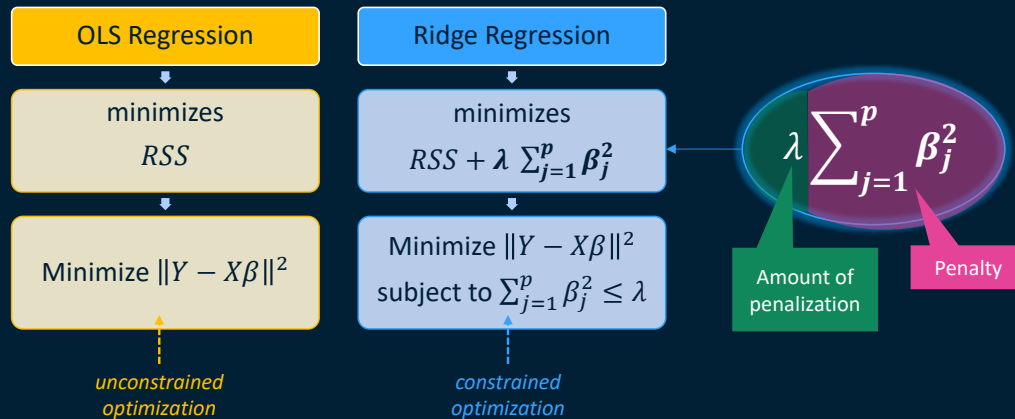
OLS estimates the parameters of a linear function by the principle of least squares. Thus, the cost function is simply minimizing the sum of the squared residuals. The OLS coefficients β s are the solution to the unconstrained optimization problem.

Lasso is a modification of linear regression. Note that the cost function has an additional factor added to the sum of the squared residuals that is a penalty term of L1 norm emphasized in the ellipsis. The model is penalized for the sum of absolute values of the weights. Thus, the absolute values of weight will be (in general) reduced, and many will tend to be zeros. A tuning parameter lambda multiplies the penalty term, allowing control over the amount of penalization. Increasing the value of lambda increases the amount of shrinkage in the parameters.

This is equivalent to a constrained form of ordinary least squares where we are constraining the sum of the absolute values of the regression coefficients so they are smaller than lambda. If the parameter lambda is small, some of the regression coefficients are zero. Due to its strong tendency of setting some parameters to zeros, it is often used to select features when we know that some features are very sparse.

Shrinking the Coefficients

L2 Type of Penalty – Ridge Regularization



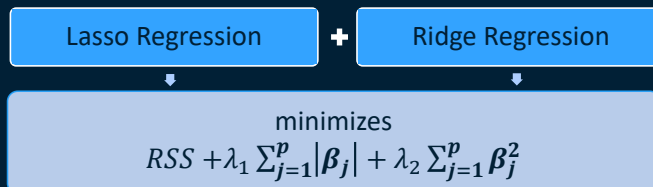
Ridge takes a step further and adds a penalty of L2 norm on the parameters β s emphasized in the ellipsis. The model is penalized for the sum of squared value of the weights. Thus, the weights not only tend to have smaller absolute values, but also tend to penalize the extremes of the weights, resulting in a group of weights that are more evenly distributed. Here also, a tuning parameter lambda controls the amount of penalization.

As with least squares, ridge regression seeks coefficient estimates that fit the data well, by making the RSS small. However, the shrinkage penalty is small when β s are close to zero, so it has the effect of shrinking the estimates of β_j toward zero but never equal to zero.

Ridge minimizes the penalized sum of squares, which is equivalent to minimization of OLS objective function subject to constraining the sum of the squared coefficients so they are smaller than lambda. Thus, the ridge coefficients β s are the solution to the constrained optimization problem.

Shrinking the Coefficients

L12 Type of Penalty – Elastic Net Regularization

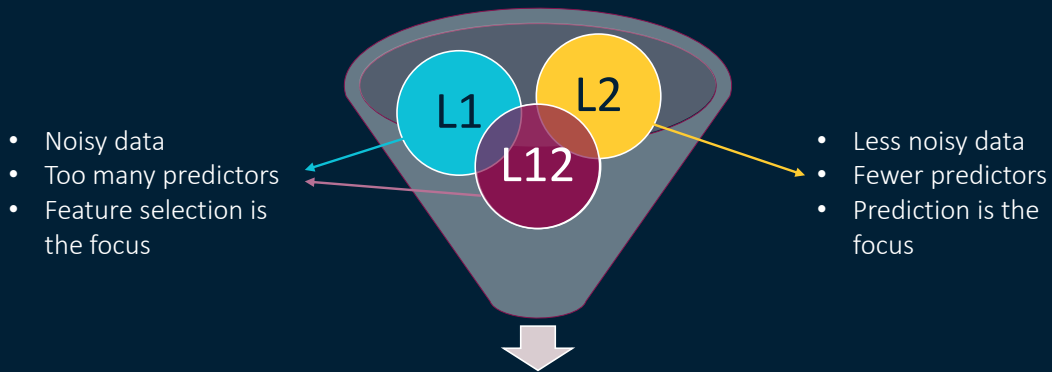


Selects features without discarding too many features

Elastic net is a hybrid of lasso and ridge, where both the absolute value penalization and squared penalization are included. Elastic net regularization is a tradeoff between L1 and L2 regularization and has a penalty that is a mix of L1 and L2 norm. It can perform the function of feature selection but still not imposing too much sparsity on the features (discarding too many features) by imposing a mixture of L1 and L2 regularization on parameters β s.

Shrinking the Coefficients

Which One to Use?



The selection of different penalties depends on the problem at hand.

With too many predictors, the signals are truly sparse - so L1 or L12 penalty can be used to select the hidden signals from noisy data, although it is almost impossible for L2 to fully recover the sparse signals.

L1 and L12 tend to give sparse weights (mostly zeros), because the L1 regularization cares equally about driving down big weights to small weights or driving small weights to zeros. If you have a lot of predictors (or features) and you suspect that not all of them are that important, it might be a really good idea to start with L1 and L12.

For problems with fewer predictors, the features are not sparse at all, so L2 regularization often outperforms L1 regularization.

L2 tends to give small but well-distributed weights, because the L2 regularization cares more about driving big weight to small weights, instead of driving small weights to zeros. If you have only a few predictors, and you are confident that all of them are relevant for predictions, L2 is a preferred regularization method.

In the prediction applications, ridge regression with L2 is more common and often recommended. However, in the case where you have many features and you want to reduce the complexity of the model by deselecting some features, you might want to impose L1 penalty or go for more of a balanced approach like elastic net.

You will need to scale your data before using these regularized linear regression methods.

L1 versus L2

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Regularization Methods

Which of the following statements is True regarding regularization methods?

- a. The sum of the absolute values of the weights is imposed as a penalty in Lasso (L1).
- b. The sum of the squared values of weights is imposed as a penalty in Ridge (L2).
- c. Elastic Net (L12) is a hybrid of Lasso (L1) and Ridge (L2).
- d. All the above.

Answer: Regularization Methods

Which of the following statements is True regarding regularization methods?

- a. The sum of the absolute values of the weights is imposed as a penalty in Lasso (L1).
- b. The sum of the squared values of weights is imposed as a penalty in Ridge (L2).
- c. Elastic Net (L12) is a hybrid of Lasso (L1) and Ridge (L2).
- ☒ d. All the above.

Answer: d

Learning Process

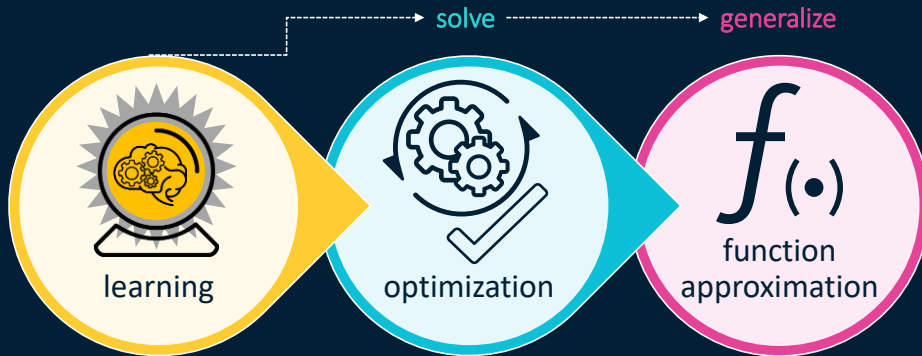
Data Difficulties and Modeling Issues



- Visualizing big data
- Processing noisy data
- Handling high dimensional data
- Dealing with distinct variability in inputs
- Using available data judiciously
- Adjusting model flexibility and complexity
- ➡ • **Understanding learning and tuning of models**
- Making models explainable

A machine learning algorithm uses a numerical optimization method that you specify to minimize the error function. It is therefore important to understand the learning process and tuning of machine learning models.

Learning Process



Machine learning involves using an algorithm to solve a mathematical problem and generalize from historical data in order to make predictions on new data. This is typically called learning a model.

Almost all machine learning algorithms can be formulated as an optimization problem. 'Optimization' comes from the same root as 'optimal', which means best. When you optimize something, you are 'making it best' given certain constraints. Learning refers to an optimization process where the parameters of the model update themselves using a mathematical function at every iteration of the training. This mathematical function is generally called an objective function and then the optimization problem is to find the extremum of this objective function.

Mathematically, solving a function optimization problem is essentially a problem of approximating the mapping function to map inputs to the target. A machine learning algorithm defines a parameterized mapping function (for example, a weighted sum of inputs) and an optimization algorithm is used to find the values of the parameters (for example, model coefficients) that minimize the error of the function. Function approximation generalizes from specific examples to a reusable mapping function that maps inputs to the target for making predictions on new examples.

Learning Process

Objective Function



$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 \cdot X_1 + \hat{\beta}_2 \cdot X_2$$

Choose parameter estimates to optimize an estimation criterion:

- Error function
- Deviance function
- Loss function
- Cost function

Minimize Squared Error function: $\sum (Y_i - \hat{Y}_i)^2$

Building models and constructing reasonable objective functions are the first step in machine learning methods. With the determined objective function, appropriate numerical optimization methods are usually used to solve the optimization problem.

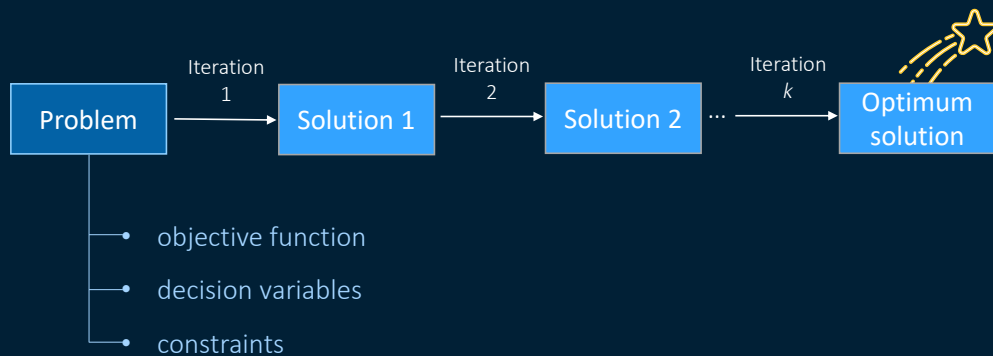
Recall that the primary task in a regression model is to obtain parameter estimates – or, in the neural network case, weight estimates – that result in accurate predictions. For example, in a linear regression model, *intercept* and *parameter estimates* are chosen to minimize the squared error between the predicted and observed target values.

Thus, in order to find the optimal solution, we need some way of measuring the quality of any solution. These estimates are best chosen by optimizing an estimation criterion which is known as an objective function. This function, taking data and model parameters as arguments, can be evaluated to return a number. Any given problem contains some parameters that can be changed. Our goal is to find values for these parameters which either maximize or minimize this number.

When you are minimizing the estimation criterion, you can also call it the *error function*, *deviance function*, *loss function*, or *cost function* - these terms are almost synonymous. They all converge to the same idea of minimizing the difference between what is observed in the data set and what is predicted by your model. The cost function is used more in optimization problems, and loss function is used in parameter estimation.

Learning Process

An Iterative Process



Optimization begins by a modeling phase, in which a given problem must be stated in terms of objective function, decision variables, and constraints. An objective function is a value that you are trying to optimize. Decision variables are inputs to your problem that your optimizer is allowed to change to try to improve the objective function value. Constraints are conditions that the decision variables must satisfy for the decision to be acceptable. This allows us to characterize the problem, and most importantly its solutions.

In general, it is not possible to directly compute a solution of an optimization problem from its formulation. Instead, almost all numerical optimization algorithms proceed in an iterative fashion. Given a current point that represents the current approximation to the solution, an optimization procedure attempts to move toward a (potentially) better solution to finally reach to an optimum solution.

So, an algorithm must have a method that will try to compute an approximate solution of the problem. By analyzing this method, it is often possible to identify how fast a method can be at getting close to a solution.

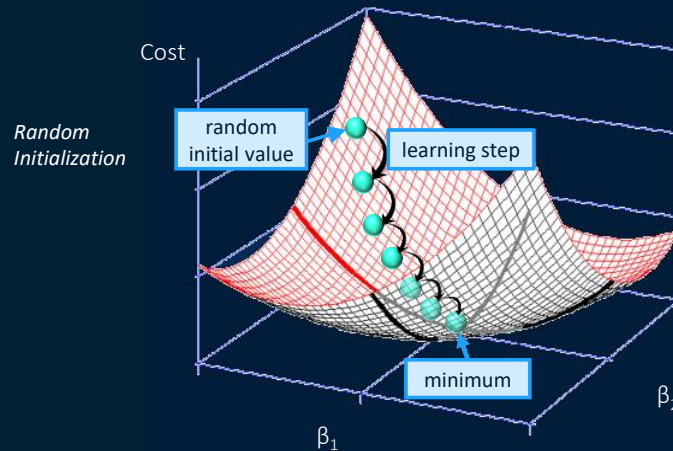
Learning Process

Gradient Descent

Gradient:
steepness of slope

Gradient Descent:
modifying parameters iteratively to minimize a cost function

- measures local gradient of error function
- goes in the direction of descending gradient



For example, gradient descent is one of the most generic optimization algorithms capable of finding optimal solutions to a wide range of problems. *Gradient* means steepness of a slope. The general idea of gradient descent is to modify parameters iteratively in order to minimize a cost function. It measures the local gradient of the error function, and it goes in the direction of descending gradient.

How big a step to take is governed by the learning rate. *Learning rate* specifies the step size for gradient descent optimization. When the gradient is zero, you have reached a minimum!

You start with random values (this is called *random initialization*), and then you improve it gradually, taking one baby step at a time, each step attempting to decrease the cost function, until the algorithm *converges* to a minimum.

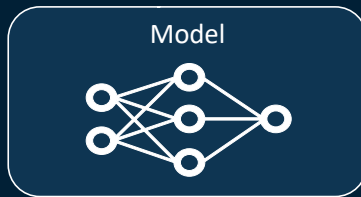
As a result, each iteration has a different set of estimated parameters. For example, in case of a neural network, a different neural network model is created in every iteration.

Learning Process

Estimating Parameters



Optimization is used for estimating parameters in a model.



Parameters are values learned through data in training process.

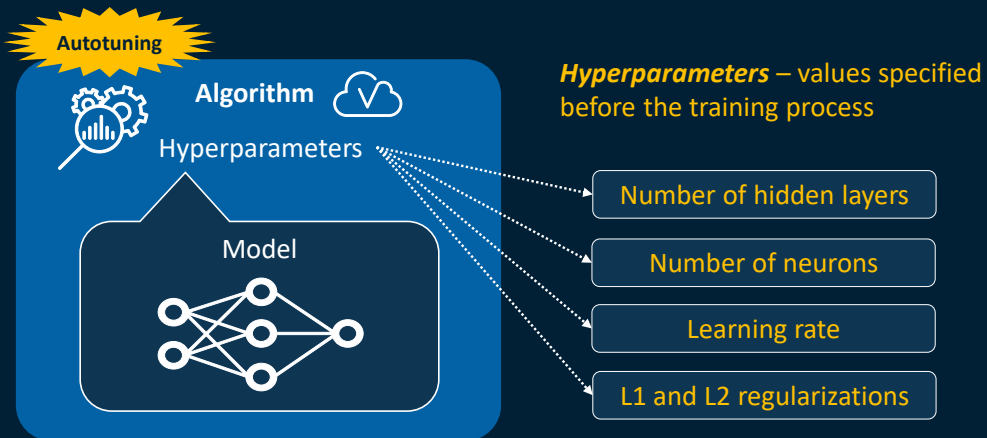
$$\begin{aligned}\text{logit}(\hat{p}) &= \hat{w}_{00} + \hat{w}_{01} H_1 + \hat{w}_{02} H_2 + \hat{w}_{03} H_3 \\ H_1 &= \tanh(\hat{w}_{10} + \hat{w}_{11} x_1 + \hat{w}_{12} x_2) \\ H_2 &= \tanh(\hat{w}_{20} + \hat{w}_{21} x_1 + \hat{w}_{22} x_2) \\ H_3 &= \tanh(\hat{w}_{30} + \hat{w}_{31} x_1 + \hat{w}_{32} x_2)\end{aligned}$$

At the core of nearly all machine learning algorithms is an optimization algorithm. For example, a global search optimization algorithm is used in a neural network model to estimate the weights and biases. These weights and biases are commonly called *parameters*. The parameters shown here are highlighted in green.

So, parameters are the values that are learned through data in the training process. They are the part of the model that is learned from historical data. Another example of parameters includes the *slope* and *intercept* in a linear regression model.

Learning Process

Tuning Hyperparameters



In addition, the process of working through a machine learning problem involves choosing the hyperparameters of a model.

A model hyperparameter is the parameter whose value is set before the training process. They cannot be learned by fitting the model to the data. Hyperparameters are not affected by training. In other words, training doesn't cause hyperparameters to change. Rather, hyperparameters affect the quality and speed of training. So, in a way these parameters are beyond or above (hyper-) the process of training and hence called hyperparameters.

For example, in a neural network model, you can specify the values for the number of hidden layers, the number of neurons per layer, the learning rate or L1/L2 regularizations, and so on - and these values cannot be estimated through training data. Thus, they are hyperparameters.

When building a model, you can set the value of hyperparameters for the model. Optimal values are determined using a trial-and-error process by adjusting or adapting their values for a particular machine learning task. This process of determining the values of hyperparameters is called hyperparameter tuning.

However, so many choices can be overwhelming. Then how do you select the best values for these hyperparameters, especially considering their varied effect on different data sets? SAS Viya automates the selection of hyperparameter values using an intelligent methodology. This capability can significantly improve the accuracy of the resulting model with no additional effort from the modeler. The autotuning feature uses optimization algorithms to select the optimal hyperparameters.

Parameters versus Hyperparameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



More about Estimation Criterion

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Learning Terminology

Which of the following terms would you use for the following description?

The difference between what is observed in the data set and what is predicted by your model.

- a. Random initialization
- b. Iterations
- c. Error function
- d. Mistake

Answer: Learning Terminology

Which of the following terms would you use for the following description?

The difference between what is observed in the data set and what is predicted by your model.

- a. Random initialization
- b. Iterations
- ☒ c. Error function
- d. Mistake

Correct answer: c

The word 'error' is used in specified sense in statistics. It doesn't mean the same thing as a 'mistake'. Mistake means wrong calculation or use of inappropriate method in the collection or analysis of data. Error means the difference between the true value and the estimated value.

Random initialization can be described as starting an optimization process with random values of weights.

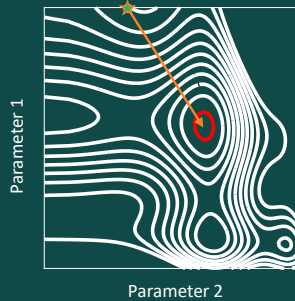
Iterations are the steps that the machine learning model takes while optimizing the model.

Question: Finding Parameter Values

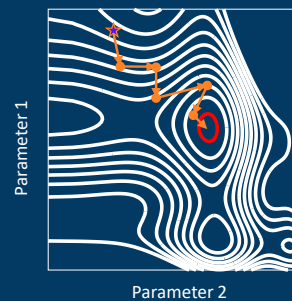


Which of the following is true for finding the parameter values for neural networks?

- a. Explicit formulas for the optimal estimates can be determined.



- b. Iterative numerical optimization methods are used.

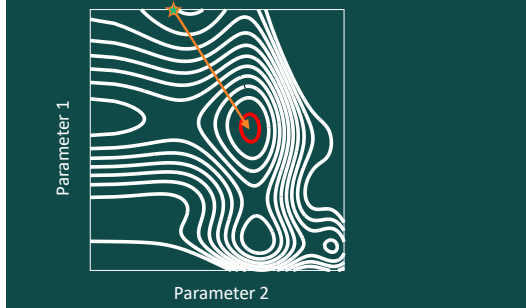


Answer: Finding Parameter Values

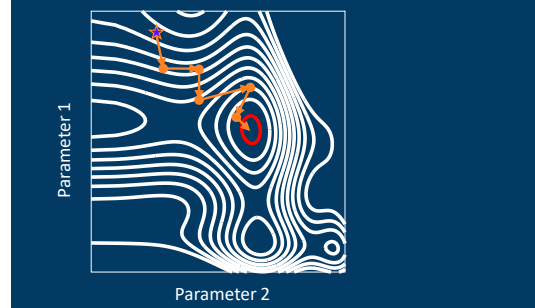


Which of the following is true for finding the parameter values for neural networks?

- a. Explicit formulas for the optimal estimates can be determined.



- b.** Iterative numerical optimization methods are used.



ANSWER: b

For some simple regression models, explicit formulas for the optimal estimates can be determined. Finding the parameter values for neural networks, however, is more difficult. Iterative numerical optimization methods are used. First, starting values are chosen. The starting values are equivalent to an initial guess at the parameter values. These values are updated to improve the estimates and reduce the error function. The updates continue until the estimates converge (in other words, there is no further progress).

Demo: Running a Neural Network Model and Tuning Its Hyperparameters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Model Interpretability

Data Difficulties and Modeling Issues

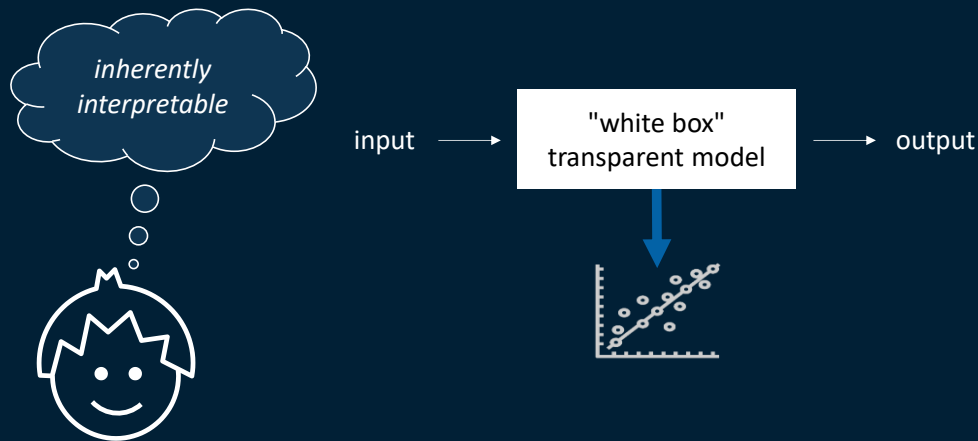


- Visualizing big data
- Processing noisy data
- Handling high dimensional data
- Dealing with distinct variability in inputs
- Using available data judiciously
- Adjusting model flexibility and complexity
- Understanding learning and tuning of models
- ➔ • **Making models explainable**

Let's discuss making models explainable, starting with white-box models.

Model Interpretability

White-Box Models



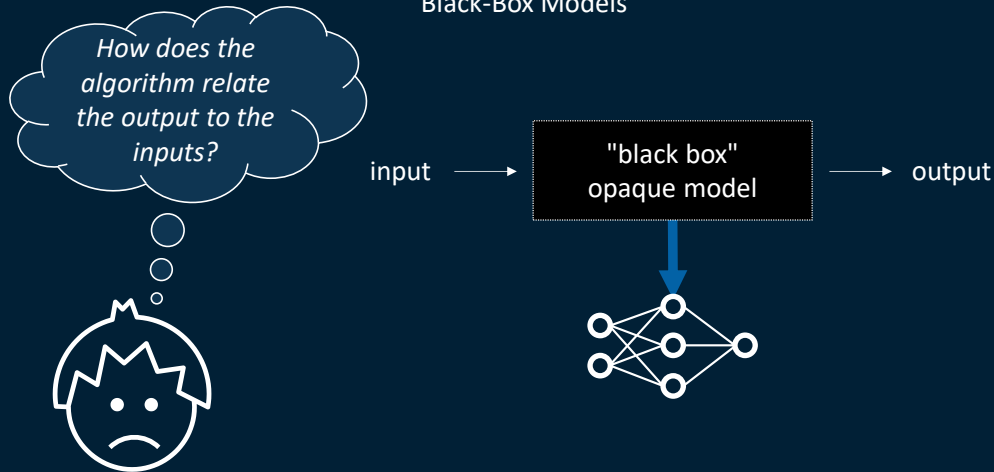
Inherently interpretable models (also called *explainable models*) incorporate interpretability directly into the model structure, and thus are self-explanatory. One type of commonly used inherently interpretable model is the generalized linear model (GLM), which includes linear and logistic regression. The coefficient estimates of GLMs directly reflect feature contributions. Hence, these models can be explained through these coefficients. Some machine learning models are simple and easy to understand.

For example, a decision tree is a transparent, or "white box," model because you know how changing the inputs will affect the predicted outcome and can make a justification for each prediction.

Inherently interpretable models generally achieve interpretability by forcing the models to use fewer features for prediction. Constraints on features can make complex models simpler and increase the model's comprehensibility to users. However, imposing these constraints can also decrease the predictive ability of the model when compared to an unrestricted model.

Model Interpretability

Black-Box Models



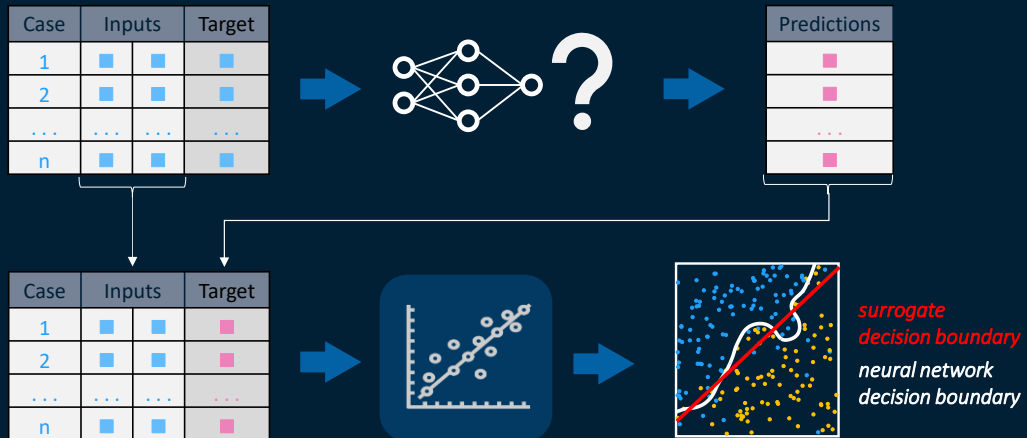
Modern machine learning algorithms can make accurate predictions by modeling the complex relationship between inputs and outputs. However, the models that are produced can be equally complex, making it difficult to understand the association between the inputs and outputs. Because of their complexity, many machine learning models are "black-box" models, producing predictions without explaining why and how they are made.

One example of a black-box machine learning model is a neural network model with one or two hidden layers. Though you can write out the equations that link every input in the model to every output, you might not be able to grasp the meaning of the connections simply by examining the equations. This has less to do with the shortcomings of the models and more to do with the shortcomings of human cognition. Often, the higher the predictive accuracy of a model, the harder it is to interpret its inner workings. So, how the algorithm creates the model is a vital problem. Algorithm transparency is about how the algorithm learns a model from the data and what type of relationships it can learn. Algorithm transparency requires only knowledge of the algorithm and not of the data or learned model.

This is where interpretability techniques come into play by providing a lens through which you can view these complex models.

Model Interpretability

Surrogate Model

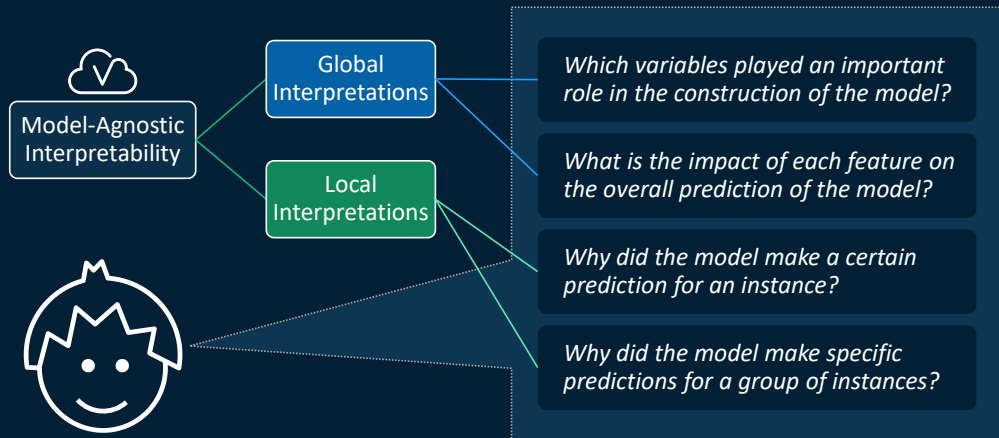


Although it is true that little insight can be gained by analyzing the actual parameters of neural networks and similar flexible models, much can be gained by analyzing the resulting predictions.

A white-box surrogate model can be created using the predictions that are produced by the pretrained neural network as a target. For example, you can approximate an inscrutable model's predictions with a regression model or a decision tree. This surrogate model approximates the behavior of the black-box model, but it is much easier to explain by examining the regression coefficients. This surrogate model does not have to work very accurately (red straight line), but it must approximate the behavior of the pretrained model's decision boundary (white curvilinear line) with some loss of generality. The idea behind a surrogate model is building an easy-to-understand model that describes the decision regions. In this way, characteristics used by the neural network to make the primary decision can be understood, even if the neural network model itself cannot be. A surrogate model can provide interpretation of the model, but it should never be assumed that the surrogate explains the true nature of how the original model makes predictions.

Model Interpretability

Understanding Black-Box Models



The idea of creating a surrogate model is the bottom line behind several model-agnostic interpretability methods that are used to explain trained supervised machine learning models such as boosted trees, forests, and neural networks. These post-hoc techniques explain predictions of these models by treating the models as black boxes and then generating explanations without inspecting the internal model parameters.

Model-agnostic interpretability techniques available in SAS Viya enable fully complex models to be interpreted either globally or locally.

Global interpretability provides explanations about the general behavior of the model over the entire population. For example, global interpretability might explain which variables played an important role in the construction of the model or describe the impact of each feature on the overall prediction of the model. Variable importance plots and partial dependence plots are global interpretability techniques.

In contrast, local interpretations promote understanding of a single data point or of a small region of the distribution, such as a cluster of input records and their corresponding predictions, or decile of predictions and their corresponding input rows. Because small sections of the conditional distribution are more likely to be linear, local explanations can be more accurate than global explanations. You can zoom in on a single instance and examine what the model predicts for this input and explain why. The global methods can be applied by taking the group of instances, treating them as if the group were the complete data set, and using the global methods with this subset.

Why Interpretability Matters

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Question: Interpreting Black-Box Models

Machine learning models are mostly black-boxes. However, SAS Viya provides several model interpretability plots that help interpret machine learning model results.

- a. True
- b. False

Answer: Interpreting Black-Box Models

Machine learning models are mostly black-boxes. However, SAS Viya provides several model interpretability plots that help interpret machine learning model results.

- ☒ a. True
- ☐ b. False

Answer: True

Model-agnostic interpretability techniques available in SAS Viya enable fully complex models to be interpreted either globally or locally.

Lesson Summary

To explore this learning resource in greater detail, please visit the course page on learn.sas.com. There, you'll find comprehensive course materials, tutorials, and updates to support your learning journey.



Where Should I Go From Here?

You are ready to take any of the following SAS courses:



Machine Learning Using
SAS Viya



Interactive Machine
Learning in SAS Viya



Supervised Machine
Learning Procedures Using
SAS Viya in SAS Studio



SAS Visual Statistics in SAS
Viya: Interactive Model
Building